

Trading Platform

Complete Source Code

Generated on 2026-02-28 | Spring Boot Backend + React Frontend

Backend – Spring Boot (Java)

■ pom.xml Trading Platform Backend/pom.xml

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0"
3     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
5     https://maven.apache.org/xsd/maven-4.0.0.xsd">
6     <modelVersion>4.0.0</modelVersion>
7     <parent>
8         <groupId>org.springframework.boot</groupId>
9         <artifactId>spring-boot-starter-parent</artifactId>
10        <version>3.2.0</version>
11        <relativePath/>
12    </parent>
13    <groupId>com.trading</groupId>
14    <artifactId>trading-platform</artifactId>
15    <version>1.0.0</version>
16    <name>Trading Platform Backend</name>
17    <description>Spring Boot Trading Platform for B.Tech Students</description>
18    <properties>
19        <java.version>17</java.version>
20    </properties>
21    <dependencies>
22        <!-- Spring Boot Web -->
23        <dependency>
24            <groupId>org.springframework.boot</groupId>
25            <artifactId>spring-boot-starter-web</artifactId>
26        </dependency>
27        <!-- Spring Data JPA -->
28        <dependency>
29            <groupId>org.springframework.boot</groupId>
30            <artifactId>spring-boot-starter-data-jpa</artifactId>
31        </dependency>
32        <!-- Validation -->
33        <dependency>
34            <groupId>org.springframework.boot</groupId>
35            <artifactId>spring-boot-starter-validation</artifactId>
36        </dependency>
37        <!-- MySQL Database Driver -->
38        <dependency>
39            <groupId>com.mysql</groupId>
40            <artifactId>mysql-connector-j</artifactId>
41            <scope>runtime</scope>
42        </dependency>
43        <!-- Lombok -->
44        <dependency>
45            <groupId>org.projectlombok</groupId>
46            <artifactId>lombok</artifactId>
47            <optional>true</optional>
48        </dependency>
49        <!-- Swagger/OpenAPI for API Documentation -->
50        <dependency>
51            <groupId>org.springdoc</groupId>
52            <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
53            <version>2.3.0</version>
54        </dependency>
55        <!-- WebClient for API calls (Alpha Vantage) -->
56        <dependency>
57            <groupId>org.springframework.boot</groupId>
58            <artifactId>spring-boot-starter-webflux</artifactId>
59        </dependency>
60        <!-- Spring Boot Test -->
61        <dependency>
62            <groupId>org.springframework.boot</groupId>
63            <artifactId>spring-boot-starter-test</artifactId>
64            <scope>test</scope>
65        </dependency>
```

```

66     </dependencies>
67     <build>
68         <plugins>
69             <plugin>
70                 <groupId>org.springframework.boot</groupId>
71                 <artifactId>spring-boot-maven-plugin</artifactId>
72                 <configuration>
73                     <excludes>
74                         <exclude>
75                             <groupId>org.projectlombok</groupId>
76                             <artifactId>lombok</artifactId>
77                         </exclude>
78                     </excludes>
79                 </configuration>
80             </plugin>
81         </plugins>
82     </build>
83 </project>

```

■ TradingPlatformApplication.java src/main/java/com/trading/TradingPlatformApplication.java

```

1 package com.trading;
2 import org.springframework.boot.SpringApplication;
3 import org.springframework.boot.autoconfigure.SpringBootApplication;
4 @SpringBootApplication
5 public class TradingPlatformApplication {
6     public static void main(String[] args) {
7         SpringApplication.run(TradingPlatformApplication.class, args);
8         System.out.println("\n=====");
9         System.out.println("■ Trading Platform Backend Started!");
10        System.out.println("=====");
11        System.out.println("■ API Base URL: http://localhost:8080/api");
12        System.out.println("■ Swagger UI: http://localhost:8080/swagger-ui.html");
13        System.out.println("■ API Docs: http://localhost:8080/v3/api-docs");
14        System.out.println("■ Database: MySQL (trading_db)");
15        System.out.println("=====\\n");
16    }
17 }

```

■ AlphaVantageConfig.java src/main/java/com/trading/config/AlphaVantageConfig.java

```

1 package com.trading.config;
2
3 import org.springframework.beans.factory.annotation.Value;
4 import org.springframework.context.annotation.Bean;
5 import org.springframework.context.annotation.Configuration;
6 import org.springframework.web.reactive.function.client.WebClient;
7
8 @Configuration
9 public class AlphaVantageConfig {
10
11     @Value("${alphavantage.api.base-url}")
12     private String baseUrl;
13
14     @Bean
15     public WebClient alphaVantageWebClient() {
16         return WebClient.builder()
17             .baseUrl(baseUrl)
18             .build();
19     }
20 }
21

```

```

1 package com.trading.config;
2 import com.trading.entity.Stock;
3 import com.trading.repository.StockRepository;
4 import lombok.RequiredArgsConstructor;
5 import org.springframework.boot.CommandLineRunner;
6 import org.springframework.stereotype.Component;
7 import java.math.BigDecimal;
8 import java.util.Arrays;
9 import java.util.List;
10 @Component
11 @RequiredArgsConstructor
12 public class DataInitializer implements CommandLineRunner {
13     private final StockRepository stockRepository;
14     @Override
15     public void run(String... args) {
16         // Initialize sample stocks if database is empty
17         if (stockRepository.count() == 0) {
18             List<Stock> stocks = Arrays.asList(
19                 createStock("AAPL", "Apple Inc.", "175.50"),
20                 createStock("GOOGL", "Alphabet Inc.", "140.25"),
21                 createStock("MSFT", "Microsoft Corporation", "380.75"),
22                 createStock("AMZN", "Amazon.com Inc.", "155.30"),
23                 createStock("TSLA", "Tesla Inc.", "245.60"),
24                 createStock("META", "Meta Platforms Inc.", "485.90"),
25                 createStock("NVDA", "NVIDIA Corporation", "725.40"),
26                 createStock("NFLX", "Netflix Inc.", "590.80"),
27                 createStock("DIS", "The Walt Disney Company", "95.20"),
28                 createStock("PYPL", "PayPal Holdings Inc.", "65.75")
29             );
30             stockRepository.saveAll(stocks);
31             System.out.println("■ Initialized " + stocks.size() + " sample stocks");
32         } else {
33             System.out.println("■ Stock data already exists (" + stockRepository.count() + "
stocks)");
34         }
35     }
36     private Stock createStock(String symbol, String companyName, String price) {
37         Stock stock = new Stock();
38         stock.setSymbol(symbol);
39         stock.setCompanyName(companyName);
40         stock.setCurrentPrice(new BigDecimal(price));
41         return stock;
42     }
43 }

```

```

1 package com.trading.config;
2 import io.swagger.v3.oas.models.OpenAPI;
3 import io.swagger.v3.oas.models.info.Contact;
4 import io.swagger.v3.oas.models.info.Info;
5 import io.swagger.v3.oas.models.info.License;
6 import io.swagger.v3.oas.models.servers.Server;
7 import org.springframework.context.annotation.Bean;
8 import org.springframework.context.annotation.Configuration;
9 import java.util.List;
10 @Configuration
11 public class SwaggerConfig {
12     @Bean
13     public OpenAPI tradingPlatformAPI() {
14         Server localServer = new Server();
15         localServer.setUrl("http://localhost:8080");
16         localServer.setDescription("Local Development Server");
17         Contact contact = new Contact();
18         contact.setName("Trading Platform Team");
19         contact.setEmail("support@tradingplatform.com");
20         License license = new License()
21             .name("Educational Use")
22             .url("https://opensource.org/licenses/MIT");
23         Info info = new Info()
24             .title("Trading Platform Backend API")
25             .version("1.0.0")
26             .description("A mini Spring Boot trading platform demonstrating core trading
operations for B.Tech students. " +
27                 "Features include user management, stock trading (buy/sell), portfolio
tracking, " +
28                 "transaction history, and live stock data integration with Finnhub API.")
29             .contact(contact)
30             .license(license);
31         return new OpenAPI()
32             .info(info)
33             .servers(List.of(localServer));
34     }
35 }

```



```

1 package com.trading.controller;
2 import com.trading.dto.ApiResponse;
3 import com.trading.dto.PortfolioResponse;
4 import com.trading.dto.TransactionResponse;
5 import com.trading.service.PortfolioService;
6 import io.swagger.v3.oas.annotations.Operation;
7 import io.swagger.v3.oas.annotations.tags.Tag;
8 import lombok.RequiredArgsConstructor;
9 import org.springframework.http.ResponseEntity;
10 import org.springframework.web.bind.annotation.*;
11 import java.math.BigDecimal;
12 import java.util.List;
13 @RestController
14 @RequestMapping("/api/portfolio")
15 @RequiredArgsConstructor
16 @CrossOrigin(origins = "*")
17 @Tag(name = "Portfolio Management", description = "APIs for viewing portfolio and transaction
history")
18 public class PortfolioController {
19     private final PortfolioService portfolioService;
20     @Operation(summary = "Get user portfolio",
21         description = "Retrieve all stock holdings for a user with profit/loss calculations")
22     @GetMapping("/{username}")
23     public ResponseEntity<ApiResponse> getUserPortfolio(@PathVariable String username) {
24         try {
25             List<PortfolioResponse> portfolio = portfolioService.getUserPortfolio(username);
26             return ResponseEntity.ok(new ApiResponse(true, "Portfolio retrieved successfully",
portfolio));
27         } catch (Exception e) {
28             return ResponseEntity.badRequest()
29                 .body(new ApiResponse(false, e.getMessage()));
30         }
31     }
32     @Operation(summary = "Get portfolio value",
33         description = "Calculate total value of all holdings")
34     @GetMapping("/{username}/value")
35     public ResponseEntity<ApiResponse> getPortfolioValue(@PathVariable String username) {
36         try {
37             BigDecimal totalValue = portfolioService.getPortfolioValue(username);
38             return ResponseEntity.ok(new ApiResponse(true, "Portfolio value calculated",
totalValue));
39         } catch (Exception e) {
40             return ResponseEntity.badRequest()
41                 .body(new ApiResponse(false, e.getMessage()));
42         }
43     }
44     @Operation(summary = "Get transaction history",
45         description = "Retrieve all buy/sell transactions for a user")
46     @GetMapping("/{username}/transactions")
47     public ResponseEntity<ApiResponse> getTransactionHistory(@PathVariable String username) {
48         try {
49             List<TransactionResponse> transactions =
portfolioService.getTransactionHistory(username);
50             return ResponseEntity.ok(new ApiResponse(true, "Transaction history retrieved",
transactions));
51         } catch (Exception e) {
52             return ResponseEntity.badRequest()
53                 .body(new ApiResponse(false, e.getMessage()));
54         }
55     }
56 }

```

```

1 package com.trading.controller;
2
3 import com.trading.dto.ApiResponse;
4 import com.trading.dto.StockResponse;
5 import com.trading.dto.alphavantage.StockPriceUpdate;
6 import com.trading.service.AlphaVantageService;
7 import com.trading.service.StockService;
8 import io.swagger.v3.oas.annotations.Operation;
9 import io.swagger.v3.oas.annotations.tags.Tag;
10 import lombok.RequiredArgsConstructor;
11 import org.springframework.http.HttpStatus;
12 import org.springframework.http.ResponseEntity;
13 import org.springframework.web.bind.annotation.*;
14 import java.util.List;
15
16 @RestController
17 @RequestMapping("/api/stocks")
18 @RequiredArgsConstructor
19 @CrossOrigin(origins = "*")
20 @Tag(name = "Stock Management", description = "APIs for viewing available stocks")
21 public class StockController {
22     private final StockService stockService;
23     private final AlphaVantageService alphaVantageService;
24
25     @Operation(summary = "Get all stocks", description = "Retrieve list of all available stocks with
current prices")
26     @GetMapping
27     public ResponseEntity<ApiResponse> getAllStocks() {
28         try {
29             List<StockResponse> stocks = stockService.getAllStocks();
30             return ResponseEntity.ok(new ApiResponse(true, "Stocks retrieved successfully", stocks));
31         } catch (Exception e) {
32             return ResponseEntity.badRequest()
33                 .body(new ApiResponse(false, e.getMessage()));
34         }
35     }
36
37     @Operation(summary = "Get stock by symbol", description = "Retrieve stock details by symbol
(e.g., AAPL, GOOGL)")
38     @GetMapping("/{symbol}")
39     public ResponseEntity<ApiResponse> getStockBySymbol(@PathVariable String symbol) {
40         try {
41             StockResponse stock = stockService.getStockResponseBySymbol(symbol);
42             return ResponseEntity.ok(new ApiResponse(true, "Stock retrieved successfully", stock));
43         } catch (Exception e) {
44             return ResponseEntity.status(404)
45                 .body(new ApiResponse(false, e.getMessage()));
46         }
47     }
48
49     @Operation(summary = "Refresh stock price", description = "Fetch latest price from Alpha Vantage
and update database")
50     @PostMapping("/{symbol}/refresh")
51     public ResponseEntity<ApiResponse> refreshStockPrice(@PathVariable String symbol) {
52         try {
53             boolean success = alphaVantageService.updateStockPrice(symbol);
54
55             if (success) {
56                 StockResponse stock = stockService.getStockResponseBySymbol(symbol);
57                 return ResponseEntity.ok(
58                     new ApiResponse(true, "Stock price refreshed successfully", stock));
59             } else {
60                 return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
61                     .body(new ApiResponse(false, "Failed to refresh stock price"));
62             }
63         } catch (Exception e) {
64             return ResponseEntity.badRequest()
65                 .body(new ApiResponse(false, e.getMessage()));
66         }
67     }
68

```

```

69     @Operation(summary = "Refresh all stock prices", description = "Fetch latest prices for all
stocks from Alpha Vantage (Rate limited)")
70     @PostMapping("/refresh-all")
71     public ResponseEntity<ApiResponse> refreshAllStockPrices() {
72         try {
73             List<StockPriceUpdate> updates = alphaVantageService.updateAllStockPrices();
74             return ResponseEntity.ok(
75                 new ApiResponse(true,
76                     "Refreshed " + updates.size() + " stock prices", updates));
77         } catch (Exception e) {
78             return ResponseEntity.badRequest()
79                 .body(new ApiResponse(false, e.getMessage()));
80         }
81     }
82
83     @Operation(summary = "Get live stock price", description = "Fetch current price from Alpha
Vantage without updating database")
84     @GetMapping("/{symbol}/live")
85     public ResponseEntity<ApiResponse> getLiveStockPrice(@PathVariable String symbol) {
86         try {
87             StockPriceUpdate livePrice = alphaVantageService.getStockPrice(symbol);
88
89             if (livePrice != null) {
90                 return ResponseEntity.ok(
91                     new ApiResponse(true, "Live price fetched", livePrice));
92             } else {
93                 return ResponseEntity.status(HttpStatus.NOT_FOUND)
94                     .body(new ApiResponse(false, "Could not fetch live price"));
95             }
96         } catch (Exception e) {
97             return ResponseEntity.badRequest()
98                 .body(new ApiResponse(false, e.getMessage()));
99         }
100     }
101 }

```

```

1 package com.trading.controller;
2 import com.trading.dto.ApiResponse;
3 import com.trading.dto.BuyStockRequest;
4 import com.trading.dto.SellStockRequest;
5 import com.trading.dto.TransactionResponse;
6 import com.trading.service.TradingService;
7 import io.swagger.v3.oas.annotations.Operation;
8 import io.swagger.v3.oas.annotations.tags.Tag;
9 import jakarta.validation.Valid;
10 import lombok.RequiredArgsConstructor;
11 import org.springframework.http.ResponseEntity;
12 import org.springframework.web.bind.annotation.*;
13 @RestController
14 @RequestMapping("/api/trading")
15 @RequiredArgsConstructor
16 @CrossOrigin(origins = "*")
17 @Tag(name = "Trading Operations", description = "Core APIs for buying and selling stocks")
18 public class TradingController {
19     private final TradingService tradingService;
20     @Operation(summary = "Buy stock",
21         description = "Purchase stocks. Validates balance, deducts amount, updates portfolio,
and saves transaction.")
22     @PostMapping("/buy")
23     public ResponseEntity<ApiResponse> buyStock(@Valid @RequestBody BuyStockRequest request) {
24         try {
25             TransactionResponse transaction = tradingService.buyStock(request);
26             return ResponseEntity.ok(new ApiResponse(true, "Stock purchased successfully",
transaction));
27         } catch (Exception e) {
28             return ResponseEntity.badRequest()
29                 .body(new ApiResponse(false, e.getMessage()));
30         }
31     }
32     @Operation(summary = "Sell stock",
33         description = "Sell stocks. Validates holdings, adds amount to balance, updates
portfolio, and saves transaction.")
34     @PostMapping("/sell")
35     public ResponseEntity<ApiResponse> sellStock(@Valid @RequestBody SellStockRequest request) {
36         try {
37             TransactionResponse transaction = tradingService.sellStock(request);
38             return ResponseEntity.ok(new ApiResponse(true, "Stock sold successfully", transaction));
39         } catch (Exception e) {
40             return ResponseEntity.badRequest()
41                 .body(new ApiResponse(false, e.getMessage()));
42         }
43     }
44 }

```

```

1 package com.trading.controller;
2 import com.trading.dto.ApiResponse;
3 import com.trading.dto.LoginRequest;
4 import com.trading.dto.UserRegistrationRequest;
5 import com.trading.entity.User;
6 import com.trading.service.UserService;
7 import io.swagger.v3.oas.annotations.Operation;
8 import io.swagger.v3.oas.annotations.tags.Tag;
9 import jakarta.validation.Valid;
10 import lombok.RequiredArgsConstructor;
11 import org.springframework.http.HttpStatus;
12 import org.springframework.http.ResponseEntity;
13 import org.springframework.web.bind.annotation.*;
14 @RestController
15 @RequestMapping("/api/users")
16 @RequiredArgsConstructor
17 @CrossOrigin(origins = "*")
18 @Tag(name = "User Management", description = "APIs for user registration and management")
19 public class UserController {
20     private final UserService userService;
21     @Operation(summary = "Register new user",
22         description = "Create a new user account with initial balance of $100,000")
23     @PostMapping("/register")
24     public ResponseEntity<ApiResponse> registerUser(@Valid @RequestBody UserRegistrationRequest
request) {
25         try {
26             User user = userService.registerUser(request);
27             return ResponseEntity.status(HttpStatus.CREATED)
28                 .body(new ApiResponse(true, "User registered successfully", user));
29         } catch (Exception e) {
30             return ResponseEntity.status(HttpStatus.BAD_REQUEST)
31                 .body(new ApiResponse(false, e.getMessage()));
32         }
33     }
34     @Operation(summary = "User login",
35         description = "Authenticate user with username and password")
36     @PostMapping("/login")
37     public ResponseEntity<ApiResponse> login(@Valid @RequestBody LoginRequest request) {
38         try {
39             User user = userService.login(request);
40             return ResponseEntity.ok(new ApiResponse(true, "Login successful", user));
41         } catch (Exception e) {
42             return ResponseEntity.status(HttpStatus.UNAUTHORIZED)
43                 .body(new ApiResponse(false, e.getMessage()));
44         }
45     }
46     @Operation(summary = "Get user by ID",
47         description = "Retrieve user details by user ID")
48     @GetMapping("/{id}")
49     public ResponseEntity<ApiResponse> getUserById(@PathVariable Long id) {
50         try {
51             User user = userService.getUserById(id);
52             return ResponseEntity.ok(new ApiResponse(true, "User retrieved successfully", user));
53         } catch (Exception e) {
54             return ResponseEntity.status(HttpStatus.NOT_FOUND)
55                 .body(new ApiResponse(false, e.getMessage()));
56         }
57     }
58     @Operation(summary = "Get user by username",
59         description = "Retrieve user details by username")
60     @GetMapping("/username/{username}")
61     public ResponseEntity<ApiResponse> getUserByUsername(@PathVariable String username) {
62         try {
63             User user = userService.getUserByUsername(username);
64             return ResponseEntity.ok(new ApiResponse(true, "User retrieved successfully", user));
65         } catch (Exception e) {
66             return ResponseEntity.status(HttpStatus.NOT_FOUND)
67                 .body(new ApiResponse(false, e.getMessage()));
68         }
69     }
70     @Operation(summary = "Get all users",

```

```

71         description = "Retrieve list of all registered users")
72     @GetMapping
73     public ResponseEntity<ApiResponse> getAllUsers() {
74         // This would be implemented in UserService
75         return ResponseEntity.ok(new ApiResponse(true, "Feature coming soon"));
76     }
77     @Operation(summary = "Delete user",
78         description = "Delete user account by ID")
79     @DeleteMapping("/{id}")
80     public ResponseEntity<ApiResponse> deleteUser(@PathVariable Long id) {
81         // This would be implemented in UserService
82         return ResponseEntity.ok(new ApiResponse(true, "Feature coming soon"));
83     }
84 }

```

■ ApiResponse.java src/main/java/com/trading/dto/ApiResponse.java

```

1 package com.trading.dto;
2 import lombok.AllArgsConstructor;
3 import lombok.Data;
4 import lombok.NoArgsConstructor;
5 @Data
6 @NoArgsConstructor
7 @AllArgsConstructor
8 public class ApiResponse {
9     private boolean success;
10    private String message;
11    private Object data;
12    public ApiResponse(boolean success, String message) {
13        this.success = success;
14        this.message = message;
15    }
16 }

```

■ BuyStockRequest.java src/main/java/com/trading/dto/BuyStockRequest.java

```

1 package com.trading.dto;
2 import jakarta.validation.constraints.Min;
3 import jakarta.validation.constraints.NotBlank;
4 import jakarta.validation.constraints.NotNull;
5 import lombok.AllArgsConstructor;
6 import lombok.Data;
7 import lombok.NoArgsConstructor;
8 @Data
9 @NoArgsConstructor
10 @AllArgsConstructor
11 public class BuyStockRequest {
12     @NotBlank(message = "Username is required")
13     private String username;
14     @NotBlank(message = "Stock symbol is required")
15     private String stockSymbol;
16     @NotNull(message = "Quantity is required")
17     @Min(value = 1, message = "Quantity must be at least 1")
18     private Integer quantity;
19 }

```

■ LoginRequest.java src/main/java/com/trading/dto/LoginRequest.java

```
1 package com.trading.dto;
2 import jakarta.validation.constraints.NotBlank;
3 import lombok.AllArgsConstructor;
4 import lombok.Data;
5 import lombok.NoArgsConstructor;
6 @Data
7 @NoArgsConstructor
8 @AllArgsConstructor
9 public class LoginRequest {
10     @NotBlank(message = "Username is required")
11     private String username;
12     @NotBlank(message = "Password is required")
13     private String password;
14 }
```

■ PortfolioResponse.java src/main/java/com/trading/dto/PortfolioResponse.java

```
1 package com.trading.dto;
2 import lombok.AllArgsConstructor;
3 import lombok.Data;
4 import lombok.NoArgsConstructor;
5 import java.math.BigDecimal;
6 @Data
7 @NoArgsConstructor
8 @AllArgsConstructor
9 public class PortfolioResponse {
10     private Long id;
11     private String stockSymbol;
12     private String companyName;
13     private Integer quantity;
14     private BigDecimal averagePrice;
15     private BigDecimal currentPrice;
16     private BigDecimal totalValue; // quantity × currentPrice
17     private BigDecimal profitLoss; // (currentPrice - averagePrice) × quantity
18 }
```

■ SellStockRequest.java src/main/java/com/trading/dto/SellStockRequest.java

```
1 package com.trading.dto;
2 import jakarta.validation.constraints.Min;
3 import jakarta.validation.constraints.NotBlank;
4 import jakarta.validation.constraints.NotNull;
5 import lombok.AllArgsConstructor;
6 import lombok.Data;
7 import lombok.NoArgsConstructor;
8 @Data
9 @NoArgsConstructor
10 @AllArgsConstructor
11 public class SellStockRequest {
12     @NotBlank(message = "Username is required")
13     private String username;
14     @NotBlank(message = "Stock symbol is required")
15     private String stockSymbol;
16     @NotNull(message = "Quantity is required")
17     @Min(value = 1, message = "Quantity must be at least 1")
18     private Integer quantity;
19 }
```

■ StockResponse.java src/main/java/com/trading/dto/StockResponse.java

```
1 package com.trading.dto;
2 import lombok.AllArgsConstructor;
3 import lombok.Data;
4 import lombok.NoArgsConstructor;
5 import java.math.BigDecimal;
6 import java.time.LocalDateTime;
7 @Data
8 @NoArgsConstructor
9 @AllArgsConstructor
10 public class StockResponse {
11     private Long id;
12     private String symbol;
13     private String companyName;
14     private BigDecimal currentPrice;
15     private LocalDateTime lastUpdated;
16 }
```

■ TransactionResponse.java src/main/java/com/trading/dto/TransactionResponse.java

```
1 package com.trading.dto;
2 import com.trading.entity.Transaction.TransactionStatus;
3 import com.trading.entity.Transaction.TransactionType;
4 import lombok.AllArgsConstructor;
5 import lombok.Data;
6 import lombok.NoArgsConstructor;
7 import java.math.BigDecimal;
8 import java.time.LocalDateTime;
9 @Data
10 @NoArgsConstructor
11 @AllArgsConstructor
12 public class TransactionResponse {
13     private Long id;
14     private String stockSymbol;
15     private String companyName;
16     private TransactionType transactionType;
17     private Integer quantity;
18     private BigDecimal pricePerShare;
19     private BigDecimal totalAmount;
20     private LocalDateTime transactionDate;
21     private TransactionStatus status;
22 }
```

■ UserRegistrationRequest.java src/main/java/com/trading/dto/UserRegistrationRequest.java

```
1 package com.trading.dto;
2 import jakarta.validation.constraints.Email;
3 import jakarta.validation.constraints.NotBlank;
4 import jakarta.validation.constraints.Size;
5 import lombok.AllArgsConstructor;
6 import lombok.Data;
7 import lombok.NoArgsConstructor;
8 @Data
9 @NoArgsConstructor
10 @AllArgsConstructor
11 public class UserRegistrationRequest {
12     @NotBlank(message = "Username is required")
13     @Size(min = 3, max = 50, message = "Username must be between 3 and 50 characters")
14     private String username;
15     @NotBlank(message = "Email is required")
16     @Email(message = "Invalid email format")
17     private String email;
18     @NotBlank(message = "Password is required")
19     @Size(min = 4, max = 20, message = "Password must be between 6 and 20 characters")
20     private String password;
21 }
```

■ GlobalQuoteResponse.java src/main/java/com/trading/dto/alphavantage/GlobalQuoteResponse.java

```
1 package com.trading.dto.alphavantage;
2
3 import com.fasterxml.jackson.annotation.JsonProperty;
4 import lombok.Data;
5
6 @Data
7 public class GlobalQuoteResponse {
8
9     @JsonProperty("Global Quote")
10     private GlobalQuote globalQuote;
11
12     @Data
13     public static class GlobalQuote {
14         @JsonProperty("01. symbol")
15         private String symbol;
16
17         @JsonProperty("05. price")
18         private String price;
19
20         @JsonProperty("07. latest trading day")
21         private String latestTradingDay;
22
23         @JsonProperty("08. previous close")
24         private String previousClose;
25
26         @JsonProperty("09. change")
27         private String change;
28
29         @JsonProperty("10. change percent")
30         private String changePercent;
31     }
32 }
33
```

■ StockPriceUpdate.java src/main/java/com/trading/dto/alphavantage/StockPriceUpdate.java

```
1 package com.trading.dto.alphavantage;
2
3 import lombok.AllArgsConstructor;
4 import lombok.Data;
5 import lombok.NoArgsConstructor;
6
7 import java.math.BigDecimal;
8 import java.time.LocalDateTime;
9
10 @Data
11 @NoArgsConstructor
12 @AllArgsConstructor
13 public class StockPriceUpdate {
14     private String symbol;
15     private BigDecimal price;
16     private BigDecimal previousClose;
17     private BigDecimal change;
18     private String changePercent;
19     private LocalDateTime updatedAt;
20     private String source; // "ALPHA_VANTAGE" or "CACHE"
21 }
22
```

■ Portfolio.java src/main/java/com/trading/entity/Portfolio.java

```
1 package com.trading.entity;
2 import jakarta.persistence.*;
3 import lombok.AllArgsConstructor;
4 import lombok.Data;
5 import lombok.NoArgsConstructor;
6 import java.math.BigDecimal;
7 @Entity
8 @Table(name = "portfolio",
9         uniqueConstraints = @UniqueConstraint(columnNames = {"user_id", "stock_id"}))
10 @Data
11 @NoArgsConstructor
12 @AllArgsConstructor
13 public class Portfolio {
14     @Id
15     @GeneratedValue(strategy = GenerationType.IDENTITY)
16     private Long id;
17     @ManyToOne(fetch = FetchType.LAZY)
18     @JoinColumn(name = "user_id", nullable = false)
19     private User user;
20     @ManyToOne(fetch = FetchType.LAZY)
21     @JoinColumn(name = "stock_id", nullable = false)
22     private Stock stock;
23     @Column(nullable = false)
24     private Integer quantity; // Number of shares owned
25     @Column(nullable = false, precision = 10, scale = 2)
26     private BigDecimal averagePrice; // Average purchase price per share
27 }
```

■ Stock.java src/main/java/com/trading/entity/Stock.java

```
1 package com.trading.entity;
2 import jakarta.persistence.*;
3 import lombok.AllArgsConstructor;
4 import lombok.Data;
5 import lombok.NoArgsConstructor;
6 import java.math.BigDecimal;
7 import java.time.LocalDateTime;
8 @Entity
9 @Table(name = "stocks")
10 @Data
11 @NoArgsConstructor
12 @AllArgsConstructor
13 public class Stock {
14     @Id
15     @GeneratedValue(strategy = GenerationType.IDENTITY)
16     private Long id;
17     @Column(nullable = false, unique = true, length = 10)
18     private String symbol; // e.g., "AAPL", "GOOGL"
19     @Column(nullable = false)
20     private String companyName; // e.g., "Apple Inc."
21     @Column(nullable = false, precision = 10, scale = 2)
22     private BigDecimal currentPrice;
23     @Column(name = "last_updated")
24     private LocalDateTime lastUpdated;
25     @PrePersist
26     @PreUpdate
27     protected void onUpdate() {
28         lastUpdated = LocalDateTime.now();
29     }
30 }
```

```

1 package com.trading.entity;
2 import jakarta.persistence.*;
3 import lombok.AllArgsConstructor;
4 import lombok.Data;
5 import lombok.NoArgsConstructor;
6 import java.math.BigDecimal;
7 import java.time.LocalDateTime;
8 @Entity
9 @Table(name = "transactions")
10 @Data
11 @NoArgsConstructor
12 @AllArgsConstructor
13 public class Transaction {
14     @Id
15     @GeneratedValue(strategy = GenerationType.IDENTITY)
16     private Long id;
17     @ManyToOne(fetch = FetchType.LAZY)
18     @JoinColumn(name = "user_id", nullable = false)
19     private User user;
20     @ManyToOne(fetch = FetchType.LAZY)
21     @JoinColumn(name = "stock_id", nullable = false)
22     private Stock stock;
23     @Column(nullable = false)
24     @Enumerated(EnumType.STRING)
25     private TransactionType transactionType;
26     @Column(nullable = false)
27     private Integer quantity;
28     @Column(nullable = false, precision = 10, scale = 2)
29     private BigDecimal pricePerShare;
30     @Column(nullable = false, precision = 15, scale = 2)
31     private BigDecimal totalAmount;
32     @Column(nullable = false)
33     private LocalDateTime transactionDate;
34     @Column(nullable = false)
35     @Enumerated(EnumType.STRING)
36     private TransactionStatus status;
37     public enum TransactionType {
38         BUY, SELL
39     }
40     public enum TransactionStatus {
41         SUCCESS, FAILED, PENDING
42     }
43     @PrePersist
44     protected void onCreate() {
45         transactionDate = LocalDateTime.now();
46     }
47 }

```

■ User.java [src/main/java/com/trading/entity/User.java](#)

```
1 package com.trading.entity;
2 import jakarta.persistence.*;
3 import lombok.AllArgsConstructor;
4 import lombok.Data;
5 import lombok.NoArgsConstructor;
6 import java.math.BigDecimal;
7 import java.time.LocalDateTime;
8 @Entity
9 @Table(name = "users")
10 @Data
11 @NoArgsConstructor
12 @AllArgsConstructor
13 public class User {
14     @Id
15     @GeneratedValue(strategy = GenerationType.IDENTITY)
16     private Long id;
17     @Column(nullable = false, unique = true, length = 50)
18     private String username;
19     @Column(nullable = false, unique = true)
20     private String email;
21     @Column(nullable = false)
22     private String password;
23     @Column(nullable = false, precision = 15, scale = 2)
24     private BigDecimal balance = new BigDecimal("100000.00"); // Initial balance $100,000
25     @Column(name = "created_at", nullable = false, updatable = false)
26     private LocalDateTime createdAt;
27     @Column(name = "updated_at")
28     private LocalDateTime updatedAt;
29     @PrePersist
30     protected void onCreate() {
31         createdAt = LocalDateTime.now();
32         updatedAt = LocalDateTime.now();
33     }
34     @PreUpdate
35     protected void onUpdate() {
36         updatedAt = LocalDateTime.now();
37     }
38 }
```

■ PortfolioRepository.java [src/main/java/com/trading/repository/PortfolioRepository.java](#)

```
1 package com.trading.repository;
2 import com.trading.entity.Portfolio;
3 import org.springframework.data.jpa.repository.JpaRepository;
4 import org.springframework.stereotype.Repository;
5 import java.util.List;
6 import java.util.Optional;
7 @Repository
8 public interface PortfolioRepository extends JpaRepository<Portfolio, Long> {
9     Optional<Portfolio> findByIdAndStockId(Long userId, Long stockId);
10     List<Portfolio> findById(Long userId);
11     List<Portfolio> findByUsername(String username);
12 }
```

■ StockRepository.java [src/main/java/com/trading/repository/StockRepository.java](#)

```
1 package com.trading.repository;
2 import com.trading.entity.Stock;
3 import org.springframework.data.jpa.repository.JpaRepository;
4 import org.springframework.stereotype.Repository;
5 import java.util.Optional;
6 @Repository
7 public interface StockRepository extends JpaRepository<Stock, Long> {
8     Optional<Stock> findBySymbol(String symbol);
9     boolean existsBySymbol(String symbol);
10 }
```

■ TransactionRepository.java src/main/java/com/trading/repository/TransactionRepository.java

```
1 package com.trading.repository;
2 import com.trading.entity.Transaction;
3 import org.springframework.data.jpa.repository.JpaRepository;
4 import org.springframework.stereotype.Repository;
5 import java.util.List;
6 @Repository
7 public interface TransactionRepository extends JpaRepository<Transaction, Long> {
8     List<Transaction> findByUserIdOrderByTransactionDateDesc(Long userId);
9     List<Transaction> findByUserUsernameOrderByTransactionDateDesc(String username);
10 }
```

■ UserRepository.java src/main/java/com/trading/repository/UserRepository.java

```
1 package com.trading.repository;
2 import com.trading.entity.User;
3 import org.springframework.data.jpa.repository.JpaRepository;
4 import org.springframework.stereotype.Repository;
5 import java.util.Optional;
6 @Repository
7 public interface UserRepository extends JpaRepository<User, Long> {
8     Optional<User> findByUsername(String username);
9     Optional<User> findByEmail(String email);
10     boolean existsByUsername(String username);
11     boolean existsByEmail(String email);
12 }
```

```

1 package com.trading.service;
2
3 import com.trading.dto.alphavantage.GlobalQuoteResponse;
4 import com.trading.dto.alphavantage.StockPriceUpdate;
5 import com.trading.entity.Stock;
6 import com.trading.repository.StockRepository;
7 import lombok.RequiredArgsConstructor;
8 import lombok.extern.slf4j.Slf4j;
9 import org.springframework.beans.factory.annotation.Value;
10 import org.springframework.stereotype.Service;
11 import org.springframework.transaction.annotation.Transactional;
12 import org.springframework.web.reactive.function.client.WebClient;
13
14 import java.math.BigDecimal;
15 import java.time.LocalDateTime;
16 import java.util.ArrayList;
17 import java.util.List;
18
19 @Service
20 @RequiredArgsConstructor
21 @Slf4j
22 public class AlphaVantageService {
23
24     private final WebClient alphaVantageWebClient;
25     private final StockRepository stockRepository;
26
27     @Value("${alphavantage.api.key}")
28     private String apiKey;
29
30     @Value("${alphavantage.api.enabled:true}")
31     private boolean apiEnabled;
32
33     /**
34      * Fetch real-time stock quote from Alpha Vantage API
35      */
36     public StockPriceUpdate fetchStockQuote(String symbol) {
37         if (!apiEnabled) {
38             log.warn("Alpha Vantage API is disabled");
39             return null;
40         }
41
42         try {
43             log.info("Fetching stock quote for: {}", symbol);
44
45             GlobalQuoteResponse response = alphaVantageWebClient.get()
46                 .uri(uriBuilder -> uriBuilder
47                     .queryParams("function", "GLOBAL_QUOTE")
48                     .queryParams("symbol", symbol)
49                     .queryParams("apikey", apiKey)
50                     .build())
51                 .retrieve()
52                 .bodyToMono(GlobalQuoteResponse.class)
53                 .block();
54
55             if (response == null || response.getGlobalQuote() == null) {
56                 log.error("No data received from Alpha Vantage for symbol: {}", symbol);
57                 return null;
58             }
59
60             GlobalQuoteResponse.GlobalQuote quote = response.getGlobalQuote();
61
62             // Check if the response contains valid data
63             if (quote.getPrice() == null || quote.getPrice().isEmpty()) {
64                 log.error("Invalid stock symbol or no data: {}", symbol);
65                 return null;
66             }
67
68             StockPriceUpdate update = new StockPriceUpdate();
69             update.setSymbol(quote.getSymbol());
70             update.setPrice(new BigDecimal(quote.getPrice()));
71             update.setPreviousClose(new BigDecimal(quote.getPreviousClose()));

```

```

72         update.setChange(new BigDecimal(quote.getChange()));
73         update.setChangePercent(quote.getChangePercent());
74         update.setUpdatedAt(LocalDateTime.now());
75         update.setSource("ALPHA_VANTAGE");
76
77         log.info("Successfully fetched price for {}: ${}", symbol, quote.getPrice());
78         return update;
79
80     } catch (Exception e) {
81         log.error("Error fetching stock quote for {}: {}", symbol, e.getMessage());
82         return null;
83     }
84 }
85
86 /**
87  * Update stock price in database
88  */
89 @Transactional
90 public boolean updateStockPrice(String symbol) {
91     try {
92         StockPriceUpdate priceUpdate = fetchStockQuote(symbol);
93
94         if (priceUpdate == null) {
95             log.error("Failed to fetch price for: {}", symbol);
96             return false;
97         }
98
99         Stock stock = stockRepository.findBySymbol(symbol.toUpperCase())
100             .orElseThrow(() -> new RuntimeException("Stock not found: " + symbol));
101
102         stock.setCurrentPrice(priceUpdate.getPrice());
103         stockRepository.save(stock);
104
105         log.info("Updated {} price to ${}", symbol, priceUpdate.getPrice());
106         return true;
107
108     } catch (Exception e) {
109         log.error("Error updating stock price for {}: {}", symbol, e.getMessage());
110         return false;
111     }
112 }
113
114 /**
115  * Update all stock prices
116  */
117 @Transactional
118 public List<StockPriceUpdate> updateAllStockPrices() {
119     List<Stock> stocks = stockRepository.findAll();
120     List<StockPriceUpdate> updates = new ArrayList<>();
121
122     log.info("Starting batch update for {} stocks", stocks.size());
123
124     for (Stock stock : stocks) {
125         try {
126             StockPriceUpdate update = fetchStockQuote(stock.getSymbol());
127
128             if (update != null) {
129                 stock.setCurrentPrice(update.getPrice());
130                 stockRepository.save(stock);
131                 updates.add(update);
132
133                 // Rate limiting: wait 12 seconds between calls (5 calls per minute)
134                 Thread.sleep(12000);
135             } else {
136                 log.warn("Skipping update for {}", stock.getSymbol());
137             }
138
139         } catch (InterruptedException e) {
140             log.error("Thread interrupted during batch update");
141             Thread.currentThread().interrupt();
142             break;
143         } catch (Exception e) {

```

```

144         log.error("Error updating {}: {}", stock.getSymbol(), e.getMessage());
145     }
146 }
147
148     log.info("Batch update completed. Updated {} out of {} stocks",
149             updates.size(), stocks.size());
150     return updates;
151 }
152
153 /**
154  * Get stock price without updating database
155  */
156 public StockPriceUpdate getStockPrice(String symbol) {
157     return fetchStockQuote(symbol);
158 }
159 }
160

```

```

1 package com.trading.service;
2 import com.trading.dto.PortfolioResponse;
3 import com.trading.dto.TransactionResponse;
4 import com.trading.entity.Portfolio;
5 import com.trading.entity.Transaction;
6 import com.trading.entity.User;
7 import com.trading.repository.PortfolioRepository;
8 import com.trading.repository.TransactionRepository;
9 import lombok.RequiredArgsConstructor;
10 import org.springframework.stereotype.Service;
11 import java.math.BigDecimal;
12 import java.util.List;
13 import java.util.stream.Collectors;
14 @Service
15 @RequiredArgsConstructor
16 public class PortfolioService {
17     private final UserService userService;
18     private final PortfolioRepository portfolioRepository;
19     private final TransactionRepository transactionRepository;
20     public List<PortfolioResponse> getUserPortfolio(String username) {
21         User user = userService.getUserByUsername(username);
22         List<Portfolio> portfolios = portfolioRepository.findByUserId(user.getId());
23         return portfolios.stream()
24             .map(this::mapToPortfolioResponse)
25             .collect(Collectors.toList());
26     }
27     public BigDecimal getPortfolioValue(String username) {
28         List<PortfolioResponse> portfolio = getUserPortfolio(username);
29         return portfolio.stream()
30             .map(PortfolioResponse::getTotalValue)
31             .reduce(BigDecimal.ZERO, BigDecimal::add);
32     }
33     public List<TransactionResponse> getTransactionHistory(String username) {
34         User user = userService.getUserByUsername(username);
35         List<Transaction> transactions = transactionRepository
36             .findByUserIdOrderByTransactionDateDesc(user.getId());
37         return transactions.stream()
38             .map(this::mapToTransactionResponse)
39             .collect(Collectors.toList());
40     }
41     private PortfolioResponse mapToPortfolioResponse(Portfolio portfolio) {
42         BigDecimal currentPrice = portfolio.getStock().getCurrentPrice();
43         BigDecimal totalValue = currentPrice.multiply(new BigDecimal(portfolio.getQuantity()));
44         BigDecimal profitLoss = currentPrice.subtract(portfolio.getAveragePrice())
45             .multiply(new BigDecimal(portfolio.getQuantity()));
46         return new PortfolioResponse(
47             portfolio.getId(),
48             portfolio.getStock().getSymbol(),
49             portfolio.getStock().getCompanyName(),
50             portfolio.getQuantity(),
51             portfolio.getAveragePrice(),
52             currentPrice,
53             totalValue,
54             profitLoss
55         );
56     }
57     private TransactionResponse mapToTransactionResponse(Transaction transaction) {
58         return new TransactionResponse(
59             transaction.getId(),
60             transaction.getStock().getSymbol(),
61             transaction.getStock().getCompanyName(),
62             transaction.getTransactionType(),
63             transaction.getQuantity(),
64             transaction.getPricePerShare(),
65             transaction.getTotalAmount(),
66             transaction.getTransactionDate(),
67             transaction.getStatus()
68         );
69     }
70 }

```

```
1 package com.trading.service;
2 import com.trading.dto.StockResponse;
3 import com.trading.entity.Stock;
4 import com.trading.repository.StockRepository;
5 import lombok.RequiredArgsConstructor;
6 import org.springframework.stereotype.Service;
7 import java.util.List;
8 import java.util.stream.Collectors;
9 @Service
10 @RequiredArgsConstructor
11 public class StockService {
12     private final StockRepository stockRepository;
13     public List<StockResponse> getAllStocks() {
14         return stockRepository.findAll().stream()
15             .map(this::mapToStockResponse)
16             .collect(Collectors.toList());
17     }
18     public Stock getStockBySymbol(String symbol) {
19         return stockRepository.findBySymbol(symbol.toUpperCase())
20             .orElseThrow(() -> new RuntimeException("Stock not found with symbol: " + symbol));
21     }
22     public StockResponse getStockResponseBySymbol(String symbol) {
23         Stock stock = getStockBySymbol(symbol);
24         return mapToStockResponse(stock);
25     }
26     private StockResponse mapToStockResponse(Stock stock) {
27         return new StockResponse(
28             stock.getId(),
29             stock.getSymbol(),
30             stock.getCompanyName(),
31             stock.getCurrentPrice(),
32             stock.getLastUpdated()
33         );
34     }
35 }
```

```

1 package com.trading.service;
2 import com.trading.dto.BuyStockRequest;
3 import com.trading.dto.SellStockRequest;
4 import com.trading.dto.TransactionResponse;
5 import com.trading.entity.*;
6 import com.trading.entity.Transaction.TransactionStatus;
7 import com.trading.entity.Transaction.TransactionType;
8 import com.trading.repository.PortfolioRepository;
9 import com.trading.repository.TransactionRepository;
10 import lombok.RequiredArgsConstructor;
11 import org.springframework.stereotype.Service;
12 import org.springframework.transaction.annotation.Transactional;
13 import java.math.BigDecimal;
14 import java.math.RoundingMode;
15 @Service
16 @RequiredArgsConstructor
17 public class TradingService {
18     private final UserService userService;
19     private final StockService stockService;
20     private final PortfolioRepository portfolioRepository;
21     private final TransactionRepository transactionRepository;
22     /**
23      * BUY STOCK LOGIC:
24      * 1. Validate user exists
25      * 2. Validate stock exists
26      * 3. Calculate total cost
27      * 4. Check if user has sufficient balance
28      * 5. Deduct amount from user balance
29      * 6. Update or create portfolio entry
30      * 7. Save transaction record
31      */
32     @Transactional
33     public TransactionResponse buyStock(BuyStockRequest request) {
34         // 1. Get user
35         User user = userService.getUserByUsername(request.getUsername());
36         // 2. Get stock
37         Stock stock = stockService.getStockBySymbol(request.getStockSymbol());
38         // 3. Calculate total cost
39         BigDecimal totalCost = stock.getCurrentPrice()
40             .multiply(new BigDecimal(request.getQuantity()));
41         // 4. Check sufficient balance
42         if (user.getBalance().compareTo(totalCost) < 0) {
43             // Save failed transaction
44             Transaction failedTransaction = createTransaction(
45                 user, stock, TransactionType.BUY,
46                 request.getQuantity(), stock.getCurrentPrice(),
47                 totalCost, TransactionStatus.FAILED
48             );
49             transactionRepository.save(failedTransaction);
50             throw new RuntimeException("Insufficient balance. Available: $" + user.getBalance() +
51                 ", Required: $" + totalCost);
52         }
53         try {
54             // 5. Deduct from user balance
55             BigDecimal newBalance = user.getBalance().subtract(totalCost);
56             userService.updateBalance(user, newBalance);
57             // 6. Update portfolio
58             Portfolio portfolio = portfolioRepository
59                 .findByIdAndStockId(user.getId(), stock.getId())
60                 .orElse(null);
61             if (portfolio == null) {
62                 // Create new portfolio entry
63                 portfolio = new Portfolio();
64                 portfolio.setUser(user);
65                 portfolio.setStock(stock);
66                 portfolio.setQuantity(request.getQuantity());
67                 portfolio.setAveragePrice(stock.getCurrentPrice());
68             } else {
69                 // Update existing portfolio entry - calculate new average price
70                 BigDecimal currentTotal = portfolio.getAveragePrice()
71                     .multiply(new BigDecimal(portfolio.getQuantity()));

```

```

72         BigDecimal newTotal = currentTotal.add(totalCost);
73         int newQuantity = portfolio.getQuantity() + request.getQuantity();
74         BigDecimal newAveragePrice = newTotal.divide(
75             new BigDecimal(newQuantity), 2, RoundingMode.HALF_UP);
76         portfolio.setQuantity(newQuantity);
77         portfolio.setAveragePrice(newAveragePrice);
78     }
79     portfolioRepository.save(portfolio);
80     // 7. Save successful transaction
81     Transaction transaction = createTransaction(
82         user, stock, TransactionType.BUY,
83         request.getQuantity(), stock.getCurrentPrice(),
84         totalCost, TransactionStatus.SUCCESS
85     );
86     Transaction savedTransaction = transactionRepository.save(transaction);
87     return mapToTransactionResponse(savedTransaction);
88 } catch (Exception e) {
89     // Save failed transaction
90     Transaction failedTransaction = createTransaction(
91         user, stock, TransactionType.BUY,
92         request.getQuantity(), stock.getCurrentPrice(),
93         totalCost, TransactionStatus.FAILED
94     );
95     transactionRepository.save(failedTransaction);
96     throw new RuntimeException("Transaction failed: " + e.getMessage());
97 }
98 }
99 /**
100  * SELL STOCK LOGIC:
101  * 1. Validate user exists
102  * 2. Validate stock exists
103  * 3. Check if user owns this stock
104  * 4. Check if user has sufficient quantity
105  * 5. Calculate total amount to receive
106  * 6. Add amount to user balance
107  * 7. Update portfolio (reduce quantity or delete if selling all)
108  * 8. Save transaction record
109  */
110 @Transactional
111 public TransactionResponse sellStock(SellStockRequest request) {
112     // 1. Get user
113     User user = userService.getUserByUsername(request.getUsername());
114     // 2. Get stock
115     Stock stock = stockService.getStockBySymbol(request.getStockSymbol());
116     // 3. Check if user owns this stock
117     Portfolio portfolio = portfolioRepository
118         .findByUserIdAndStockId(user.getId(), stock.getId())
119         .orElseThrow(() -> new RuntimeException("You don't own any shares of " +
stock.getSymbol()));
120     // 4. Check if user has sufficient quantity
121     if (portfolio.getQuantity() < request.getQuantity()) {
122         throw new RuntimeException("Insufficient shares. You own " + portfolio.getQuantity() +
123             " shares, trying to sell " + request.getQuantity());
124     }
125     // 5. Calculate total amount to receive
126     BigDecimal totalAmount = stock.getCurrentPrice()
127         .multiply(new BigDecimal(request.getQuantity()));
128     try {
129         // 6. Add to user balance
130         BigDecimal newBalance = user.getBalance().add(totalAmount);
131         userService.updateBalance(user, newBalance);
132         // 7. Update portfolio
133         int newQuantity = portfolio.getQuantity() - request.getQuantity();
134         if (newQuantity == 0) {
135             // Sold all shares, delete portfolio entry
136             portfolioRepository.delete(portfolio);
137         } else {
138             // Update quantity
139             portfolio.setQuantity(newQuantity);
140             portfolioRepository.save(portfolio);
141         }
142         // 8. Save successful transaction

```

```

143         Transaction transaction = createTransaction(
144             user, stock, TransactionType.SELL,
145             request.getQuantity(), stock.getCurrentPrice(),
146             totalAmount, TransactionStatus.SUCCESS
147         );
148         Transaction savedTransaction = transactionRepository.save(transaction);
149         return mapToTransactionResponse(savedTransaction);
150     } catch (Exception e) {
151         // Save failed transaction
152         Transaction failedTransaction = createTransaction(
153             user, stock, TransactionType.SELL,
154             request.getQuantity(), stock.getCurrentPrice(),
155             totalAmount, TransactionStatus.FAILED
156         );
157         transactionRepository.save(failedTransaction);
158         throw new RuntimeException("Transaction failed: " + e.getMessage());
159     }
160 }
161 private Transaction createTransaction(User user, Stock stock, TransactionType type,
162                                     Integer quantity, BigDecimal pricePerShare,
163                                     BigDecimal totalAmount, TransactionStatus status) {
164     Transaction transaction = new Transaction();
165     transaction.setUser(user);
166     transaction.setStock(stock);
167     transaction.setTransactionType(type);
168     transaction.setQuantity(quantity);
169     transaction.setPricePerShare(pricePerShare);
170     transaction.setTotalAmount(totalAmount);
171     transaction.setStatus(status);
172     return transaction;
173 }
174 private TransactionResponse mapToTransactionResponse(Transaction transaction) {
175     return new TransactionResponse(
176         transaction.getId(),
177         transaction.getStock().getSymbol(),
178         transaction.getStock().getCompanyName(),
179         transaction.getTransactionType(),
180         transaction.getQuantity(),
181         transaction.getPricePerShare(),
182         transaction.getTotalAmount(),
183         transaction.getTransactionDate(),
184         transaction.getStatus()
185     );
186 }
187 }

```

```

1 package com.trading.service;
2 import com.trading.dto.LoginRequest;
3 import com.trading.dto.UserRegistrationRequest;
4 import com.trading.entity.User;
5 import com.trading.repository.UserRepository;
6 import lombok.RequiredArgsConstructor;
7 import org.springframework.stereotype.Service;
8 import org.springframework.transaction.annotation.Transactional;
9 import java.math.BigDecimal;
10 @Service
11 @RequiredArgsConstructor
12 public class UserService {
13     private final UserRepository userRepository;
14     @Transactional
15     public User registerUser(UserRegistrationRequest request) {
16         // Check if username already exists
17         if (userRepository.existsByUsername(request.getUsername())) {
18             throw new RuntimeException("Username already exists");
19         }
20         // Check if email already exists
21         if (userRepository.existsByEmail(request.getEmail())) {
22             throw new RuntimeException("Email already registered");
23         }
24         // Create user with initial balance of $100,000
25         User user = new User();
26         user.setUsername(request.getUsername());
27         user.setEmail(request.getEmail());
28         user.setPassword(request.getPassword()); // In production, hash this with BCrypt!
29         user.setBalance(new BigDecimal("100000.00")); // Initial balance
30         return userRepository.save(user);
31     }
32     public User login(LoginRequest request) {
33         User user = userRepository.findByUsername(request.getUsername())
34             .orElseThrow(() -> new RuntimeException("Invalid username or password"));
35         // Validate password (in production, use BCrypt password matching)
36         if (!user.getPassword().equals(request.getPassword())) {
37             throw new RuntimeException("Invalid username or password");
38         }
39         return user;
40     }
41     public User getUserById(Long id) {
42         return userRepository.findById(id)
43             .orElseThrow(() -> new RuntimeException("User not found with ID: " + id));
44     }
45     public User getUserByUsername(String username) {
46         return userRepository.findByUsername(username)
47             .orElseThrow(() -> new RuntimeException("User not found with username: " +
username));
48     }
49     @Transactional
50     public void updateBalance(User user, BigDecimal newBalance) {
51         user.setBalance(newBalance);
52         userRepository.save(user);
53     }
54 }

```

■ application.properties src/main/resources/application.properties

```
1 # Server Configuration
2 server.port=8080
3 # MySQL Database Configuration
4 spring.datasource.url=jdbc:mysql://localhost:3306/trading_db?createDatabaseIfNotExist=true&useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC
5 spring.datasource.driverClassName=com.mysql.cj.jdbc.Driver
6 spring.datasource.username=root
7 spring.datasource.password=pranayraju
8 # JPA/Hibernate Configuration
9 spring.jpa.database-platform=org.hibernate.dialect.MySQLDialect
10 spring.jpa.hibernate.ddl-auto=update
11 spring.jpa.show-sql=true
12 spring.jpa.properties.hibernate.format_sql=true
13 spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect
14 # Logging
15 logging.level.org.springframework.web=INFO
16 logging.level.com.trading=DEBUG
17 # Application Name
18 spring.application.name=Trading Platform Backend
19
20 # Alpha Vantage API Configuration
21 alphavantage.api.key=698ed00dd679c3.41538682
22 alphavantage.api.base-url=https://www.alphavantage.co/query
23 alphavantage.api.enabled=true
24
25 # Logging for Alpha Vantage
26 logging.level.com.trading.service.AlphaVantageService=INFO
```

■ TradingPlatformBackendApplicationTests.java

src/test/java/com/trading/TradingPlatformBackendApplicationTests.java

```
1 package com.trading;
2
3 import org.junit.jupiter.api.Test;
4 import org.springframework.boot.test.context.SpringBootTest;
5
6 @SpringBootTest
7 class TradingPlatformBackendApplicationTests {
8
9     @Test
10     void contextLoads() {
11     }
12
13 }
14
```

■ Frontend – React (Vite)

■ index.html trading-frontend/index.html

```
1 <!doctype html>
2 <html lang="en">
3   <head>
4     <meta charset="UTF-8" />
5     <link rel="icon" type="image/svg+xml" href="/vite.svg" />
6     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
7     <meta name="description" content="MRU Trading Platform - Full-Stack Trading Application" />
8     <link rel="preconnect" href="https://fonts.googleapis.com">
9     <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
10    <link href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700;800&display=s
wap" rel="stylesheet">
11    <title>MRU Trading Platform</title>
12  </head>
13  <body>
14    <div id="root"></div>
15    <script type="module" src="/src/main.jsx"></script>
16  </body>
17 </html>
18
```

■ vite.config.js trading-frontend/vite.config.js

```
1 import { defineConfig } from 'vite'
2 import react from '@vitejs/plugin-react'
3
4 // https://vitejs.dev/config/
5 export default defineConfig({
6   plugins: [react()],
7   server: {
8     port: 5173,
9     open: true
10  }
11 })
12
```

```
1 {
2   "name": "trading-frontend",
3   "private": true,
4   "version": "0.0.0",
5   "type": "module",
6   "scripts": {
7     "dev": "vite",
8     "build": "vite build",
9     "preview": "vite preview"
10  },
11  "dependencies": {
12    "react": "^18.3.1",
13    "react-dom": "^18.3.1",
14    "react-router-dom": "^6.22.0",
15    "axios": "^1.6.7",
16    "chart.js": "^4.4.1",
17    "react-chartjs-2": "^5.2.0",
18    "lucide-react": "^0.323.0",
19    "react-hot-toast": "^2.4.1"
20  },
21  "devDependencies": {
22    "@types/react": "^18.3.12",
23    "@types/react-dom": "^18.3.1",
24    "@vitejs/plugin-react": "^4.3.3",
25    "vite": "^5.4.10"
26  }
27 }
28
```

```

1 import { BrowserRouter as Router, Routes, Route, Navigate } from 'react-router-dom';
2 import { AuthProvider, useAuth } from './context/AuthContext';
3 import { Toaster } from 'react-hot-toast';
4 import Login from './components/auth/Login';
5 import Register from './components/auth/Register';
6 import Dashboard from './components/dashboard/Dashboard';
7 import Navbar from './components/dashboard/Navbar';
8 import Market from './components/market/Market';
9 import Portfolio from './components/portfolio/Portfolio';
10 import Transactions from './components/transactions/Transactions';
11 import './styles/index.css';
12
13 // Protected Route Component
14 const ProtectedRoute = ({ children }) => {
15   const { isAuthenticated, loading } = useAuth();
16
17   if (loading) {
18     return (
19       <div className="flex-center" style={{ minHeight: '100vh' }}>
20         <div className="spinner"></div>
21       </div>
22     );
23   }
24
25   return isAuthenticated ? children : <Navigate to="/login" replace />;
26 };
27
28 // Layout Component
29 const Layout = ({ children }) => {
30   return (
31     <>
32       <Navbar />
33       {children}
34     </>
35   );
36 };
37
38 function App() {
39   return (
40     <AuthProvider>
41       <Router>
42         <Routes>
43           {/* Public Routes */}
44           <Route path="/login" element={<Login />} />
45           <Route path="/register" element={<Register />} />
46
47           {/* Protected Routes */}
48           <Route
49             path="/dashboard"
50             element={
51               <ProtectedRoute>
52                 <Layout>
53                   <Dashboard />
54                 </Layout>
55               </ProtectedRoute>
56             }
57           />
58           <Route
59             path="/market"
60             element={
61               <ProtectedRoute>
62                 <Layout>
63                   <Market />
64                 </Layout>
65               </ProtectedRoute>
66             }
67           />
68           <Route
69             path="/portfolio"
70             element={
71               <ProtectedRoute>

```

```

72         <Layout>
73             <Portfolio />
74         </Layout>
75     </ProtectedRoute>
76 }
77 />
78 <Route
79     path="/transactions"
80     element={
81         <ProtectedRoute>
82             <Layout>
83                 <Transactions />
84             </Layout>
85         </ProtectedRoute>
86     }
87 />
88
89     {/* Default Route */}
90     <Route path="/" element={<Navigate to="/dashboard" replace />} />
91     <Route path="*" element={<Navigate to="/dashboard" replace />} />
92 </Routes>
93
94     {/* Toast Notifications */}
95     <Toaster
96         position="top-right"
97         toastOptions={{
98             duration: 3000,
99             style: {
100                 background: '#1a1f3a',
101                 color: '#f8fafc',
102                 border: '1px solid rgba(255, 255, 255, 0.1)',
103                 borderRadius: '0.75rem',
104                 padding: '1rem',
105             },
106             success: {
107                 iconTheme: {
108                     primary: '#10b981',
109                     secondary: '#f8fafc',
110                 },
111             },
112             error: {
113                 iconTheme: {
114                     primary: '#ef4444',
115                     secondary: '#f8fafc',
116                 },
117             },
118         }}
119     />
120 </Router>
121 </AuthProvider>
122 );
123 }
124
125 export default App;
126

```

main.jsx src/main.jsx

```

1 import React from 'react';
2 import ReactDOM from 'react-dom/client';
3 import App from './App.jsx';
4
5 ReactDOM.createRoot(document.getElementById('root')).render(
6     <React.StrictMode>
7     <App />
8 </React.StrictMode>
9 );
10

```

```
1 .auth-container {
2   min-height: 100vh;
3   display: flex;
4   align-items: center;
5   justify-content: center;
6   padding: var(--space-lg);
7   background: linear-gradient(135deg, var(--color-bg-primary) 0%, var(--color-bg-secondary) 100%);
8   position: relative;
9   overflow: hidden;
10 }
11
12 .auth-container::before {
13   content: '';
14   position: absolute;
15   top: -50%;
16   right: -50%;
17   width: 100%;
18   height: 100%;
19   background: radial-gradient(circle, rgba(99, 102, 241, 0.1) 0%, transparent 70%);
20   animation: pulse 4s ease-in-out infinite;
21 }
22
23 .auth-card {
24   width: 100%;
25   max-width: 450px;
26   padding: var(--space-2xl);
27   animation: fadeIn 0.5s ease-out;
28   position: relative;
29   z-index: 1;
30 }
31
32 .auth-header {
33   text-align: center;
34   margin-bottom: var(--space-2xl);
35 }
36
37 .auth-icon {
38   display: inline-flex;
39   align-items: center;
40   justify-content: center;
41   width: 80px;
42   height: 80px;
43   background: linear-gradient(135deg, var(--color-primary), var(--color-accent));
44   border-radius: var(--radius-xl);
45   margin-bottom: var(--space-lg);
46   box-shadow: var(--shadow-glow);
47   color: white;
48 }
49
50 .auth-header h1 {
51   margin-bottom: var(--space-sm);
52   font-size: var(--font-size-3xl);
53 }
54
55 .auth-header p {
56   color: var(--color-text-tertiary);
57   font-size: var(--font-size-base);
58 }
59
60 .auth-form {
61   margin-bottom: var(--space-lg);
62 }
63
64 .auth-form .form-label {
65   display: flex;
66   align-items: center;
67   gap: var(--space-xs);
68 }
69
70 .auth-footer {
71   text-align: center;
```

```
72     padding-top: var(--space-lg);
73     border-top: 1px solid var(--color-divider);
74 }
75
76 .auth-footer p {
77     color: var(--color-text-tertiary);
78     font-size: var(--font-size-sm);
79 }
80
81 .auth-link {
82     color: var(--color-primary);
83     text-decoration: none;
84     font-weight: 600;
85     transition: color var(--transition-fast);
86 }
87
88 .auth-link:hover {
89     color: var(--color-primary-light);
90     text-decoration: underline;
91 }
92
93 @media (max-width: 768px) {
94     .auth-card {
95         padding: var(--space-xl);
96     }
97
98     .auth-icon {
99         width: 60px;
100        height: 60px;
101    }
102 }
```

■ Login.jsx src/components/auth/Login.jsx

```
1 import { useState } from 'react';
2 import { Link, useNavigate } from 'react-router-dom';
3 import { useAuth } from '../../context/AuthContext';
4 import toast from 'react-hot-toast';
5 import { LogIn, Mail, Lock, User } from 'lucide-react';
6 import './Auth.css';
7
8 const Login = () => {
9   const navigate = useNavigate();
10   const { login } = useAuth();
11   const [formData, setFormData] = useState({
12     username: '',
13     password: '',
14   });
15   const [loading, setLoading] = useState(false);
16
17   const handleChange = (e) => {
18     setFormData({
19       ...formData,
20       [e.target.name]: e.target.value,
21     });
22   };
23
24   const handleSubmit = async (e) => {
25     e.preventDefault();
26     setLoading(true);
27
28     try {
29       const response = await login(formData);
30       if (response.success) {
31         toast.success('Login successful! Welcome back!');
32         navigate('/dashboard');
33       } else {
34         toast.error(response.message || 'Login failed');
35       }
36     } catch (error) {
37       toast.error(error.message || 'Invalid credentials');
38     } finally {
39       setLoading(false);
40     }
41   };
42
43   return (
44     <div className="auth-container">
45       <div className="auth-card glass-card">
46         <div className="auth-header">
47           <div className="auth-icon">
48             <LogIn size={40} />
49           </div>
50           <h1>Welcome Back</h1>
51           <p>Sign in to your trading account</p>
52         </div>
53
54         <form onSubmit={handleSubmit} className="auth-form">
55           <div className="form-group">
56             <label className="form-label">
57               <User size={16} />
58               Username
59             </label>
60             <input
61
62               type="text"
63               name="username"
64               className="form-input"
65               placeholder="Enter your username"
66               value={formData.username}
67               onChange={handleChange}
68               required
69             />
70           </div>
71
72           <div className="form-group">
```

```

72         <label className="form-label">
73             <Lock size={16} />
74             Password
75         </label>
76         <input
77             type="password"
78             name="password"
79             className="form-input"
80             placeholder="Enter your password"
81             value={formData.password}
82             onChange={handleChange}
83             required
84         />
85     </div>
86
87     <button type="submit" className="btn btn-primary btn-lg" disabled={loading}>
88         {loading ? (
89             <>
90                 <div className="spinner" style={{ width: '20px', height: '20px',
borderWidth: '2px' }}></div>
91                 Signing in...
92             </>
93         ) : (
94             <>
95                 <LogIn size={20} />
96                 Sign In
97             </>
98         )}
99     </button>
100 </form>
101
102     <div className="auth-footer">
103         <p>
104             Don't have an account?{' '}
105             <Link to="/register" className="auth-link">
106                 Create one now
107             </Link>
108         </p>
109     </div>
110 </div>
111 </div>
112 );
113 };
114
115 export default Login;
116

```

■ Register.jsx src/components/auth/Register.jsx

```
1 import { useState } from 'react';
2 import { Link, useNavigate } from 'react-router-dom';
3 import { useAuth } from '../../../context/AuthContext';
4 import toast from 'react-hot-toast';
5 import { UserPlus, Mail, Lock, User } from 'lucide-react';
6 import './Auth.css';
7
8 const Register = () => {
9   const navigate = useNavigate();
10   const { register } = useAuth();
11   const [formData, setFormData] = useState({
12     username: '',
13     email: '',
14     password: '',
15   });
16   const [loading, setLoading] = useState(false);
17
18   const handleChange = (e) => {
19     setFormData({
20       ...formData,
21       [e.target.name]: e.target.value,
22     });
23   };
24
25   const handleSubmit = async (e) => {
26     e.preventDefault();
27     setLoading(true);
28
29     try {
30       const response = await register(formData);
31       if (response.success) {
32         toast.success('Registration successful! You can now login with $100,000 initial
balance!');
33         navigate('/login');
34       } else {
35         toast.error(response.message || 'Registration failed');
36       }
37     } catch (error) {
38       toast.error(error.message || 'Registration failed. Username or email may already
exist.');
```

```
39     } finally {
40       setLoading(false);
41     }
42   };
43
44   return (
45     <div className="auth-container">
46       <div className="auth-card glass-card">
47         <div className="auth-header">
48           <div className="auth-icon">
49             <UserPlus size={40} />
50           </div>
51           <h1>Create Account</h1>
52           <p>Start trading with $100,000 initial balance</p>
53         </div>
54
55         <form onSubmit={handleSubmit} className="auth-form">
56           <div className="form-group">
57             <label className="form-label">
58               <User size={16} />
59               Username
60             </label>
61
62             <input
63               type="text"
64               name="username"
65               className="form-input"
66               placeholder="Choose a username"
67               value={formData.username}
68               onChange={handleChange}
69               minLength={3}
70               maxLength={50}
71             />
72           </div>
73
74           <div className="form-group">
75             <label className="form-label">
76               <Mail size={16} />
77               Email
78             </label>
79
80             <input
81               type="text"
82               name="email"
83               className="form-input"
84               placeholder="Enter your email"
85               value={formData.email}
86               onChange={handleChange}
87               minLength={5}
88               maxLength={50}
89             />
90           </div>
91
92           <div className="form-group">
93             <label className="form-label">
94               <Lock size={16} />
95               Password
96             </label>
97
98             <input
99               type="password"
100              name="password"
101              className="form-input"
102              placeholder="Enter your password"
103              value={formData.password}
104              onChange={handleChange}
105              minLength={6}
106              maxLength={50}
107            />
108          </div>
109
110          <div style="text-align: right; margin-top: 10px;">
111            <button type="button" className="button" href="#"><Link to Login></button>
112            <button type="submit" className="button">Register</button>
113          </div>
114        </form>
115      </div>
116    </div>
117  );
118};
```

```

70         required
71     />
72 </div>
73
74 <div className="form-group">
75     <label className="form-label">
76         <Mail size={16} />
77         Email
78     </label>
79     <input
80         type="email"
81         name="email"
82         className="form-input"
83         placeholder="Enter your email"
84         value={formData.email}
85         onChange={handleChange}
86         required
87     />
88 </div>
89
90 <div className="form-group">
91     <label className="form-label">
92         <Lock size={16} />
93         Password
94     </label>
95     <input
96         type="password"
97         name="password"
98         className="form-input"
99         placeholder="Create a password"
100        value={formData.password}
101        onChange={handleChange}
102        minLength={6}
103        maxLength={20}
104        required
105    />
106 </div>
107
108 <button type="submit" className="btn btn-primary btn-lg" disabled={loading}>
109     {loading ? (
110         <>
111             <div className="spinner" style={{ width: '20px', height: '20px',
borderWidth: '2px' }}></div>
112             Creating account...
113         </>
114     ) : (
115         <>
116             <UserPlus size={20} />
117             Create Account
118         </>
119     )}
120 </button>
121 </form>
122
123 <div className="auth-footer">
124     <p>
125         Already have an account?{' '}
126         <Link to="/login" className="auth-link">
127             Sign in here
128         </Link>
129     </p>
130 </div>
131 </div>
132 </div>
133 );
134 };
135
136 export default Register;
137

```

```

1 .dashboard {
2   padding: var(--space-xl);
3   max-width: 1400px;
4   margin: 0 auto;
5   animation: fadeIn 0.5s ease-out;
6 }
7
8 .dashboard-loading {
9   display: flex;
10  flex-direction: column;
11  align-items: center;
12  justify-content: center;
13  min-height: 60vh;
14  gap: var(--space-lg);
15 }
16
17 .dashboard-header {
18   margin-bottom: var(--space-2xl);
19 }
20
21 .dashboard-header h1 {
22   margin-bottom: var(--space-sm);
23   background: linear-gradient(135deg, var(--color-primary), var(--color-accent));
24   -webkit-background-clip: text;
25   -webkit-text-fill-color: transparent;
26   background-clip: text;
27 }
28
29 .dashboard-header p {
30   color: var(--color-text-tertiary);
31   font-size: var(--font-size-lg);
32 }
33
34 .stats-grid {
35   display: grid;
36   grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
37   gap: var(--space-lg);
38   margin-bottom: var(--space-2xl);
39 }
40
41 .stat-card {
42   display: flex;
43   align-items: center;
44   gap: var(--space-lg);
45   padding: var(--space-xl);
46   transition: all var(--transition-base);
47 }
48
49 .stat-card:hover {
50   transform: translateY(-4px);
51 }
52
53 .stat-icon {
54   display: flex;
55   align-items: center;
56   justify-content: center;
57   width: 60px;
58   height: 60px;
59   border-radius: var(--radius-lg);
60   color: white;
61   flex-shrink: 0;
62 }
63
64 .stat-content {
65   flex: 1;
66 }
67
68 .stat-label {
69   font-size: var(--font-size-sm);
70   color: var(--color-text-tertiary);
71   margin-bottom: var(--space-xs);

```

```

72     text-transform: uppercase;
73     letter-spacing: 0.5px;
74 }
75
76 .stat-value {
77     font-size: var(--font-size-2xl);
78     font-weight: 800;
79     color: var(--color-text-primary);
80 }
81
82 .dashboard-content {
83     display: grid;
84     grid-template-columns: repeat(auto-fit, minmax(400px, 1fr));
85     gap: var(--space-xl);
86 }
87
88 .dashboard-section h2 {
89     display: flex;
90     align-items: center;
91     gap: var(--space-md);
92     margin-bottom: var(--space-lg);
93     font-size: var(--font-size-xl);
94 }
95
96 .top-stocks,
97 .transactions-list {
98     display: flex;
99     flex-direction: column;
100    gap: var(--space-md);
101 }
102
103 .stock-item {
104     display: flex;
105     align-items: center;
106     justify-content: space-between;
107     padding: var(--space-lg);
108     transition: all var(--transition-fast);
109 }
110
111 .stock-item:hover {
112     transform: translateX(4px);
113 }
114
115 .stock-info h4 {
116     font-size: var(--font-size-lg);
117     margin-bottom: var(--space-xs);
118 }
119
120 .stock-info p {
121     font-size: var(--font-size-sm);
122     color: var(--color-text-tertiary);
123 }
124
125 .stock-price .price {
126     font-size: var(--font-size-xl);
127     font-weight: 700;
128     color: var(--color-success);
129 }
130
131 .transaction-item {
132     display: flex;
133     align-items: center;
134     gap: var(--space-lg);
135     padding: var(--space-lg);
136     transition: all var(--transition-fast);
137 }
138
139 .transaction-item:hover {
140     transform: translateX(4px);
141 }
142
143 .transaction-details {

```

```

144     flex: 1;
145 }
146
147 .transaction-details h4 {
148     font-size: var(--font-size-base);
149     margin-bottom: var(--space-xs);
150 }
151
152 .transaction-details p {
153     font-size: var(--font-size-sm);
154     color: var(--color-text-tertiary);
155 }
156
157 .transaction-amount {
158     font-size: var(--font-size-lg);
159     font-weight: 700;
160 }
161
162 @media (max-width: 1024px) {
163     .dashboard-content {
164         grid-template-columns: 1fr;
165     }
166 }
167
168 @media (max-width: 768px) {
169     .dashboard {
170         padding: var(--space-md);
171     }
172
173     .stats-grid {
174         grid-template-columns: 1fr;
175     }
176
177     .stat-card {
178         padding: var(--space-lg);
179     }
180
181     .stat-icon {
182         width: 50px;
183         height: 50px;
184     }
185
186     .stat-value {
187         font-size: var(--font-size-xl);
188     }
189 }

```

■ Dashboard.jsx src/components/dashboard/Dashboard.jsx

```
1 import { useState, useEffect } from 'react';
2 import { useAuth } from '../../../context/AuthContext';
3 import portfolioService from '../../../services/portfolioService';
4 import stockService from '../../../services/stockService';
5 import { Wallet, TrendingUp, TrendingDown, Activity, DollarSign } from 'lucide-react';
6 import toast from 'react-hot-toast';
7 import './Dashboard.css';
8
9 const Dashboard = () => {
10   const { user, refreshUser } = useAuth();
11   const [portfolioValue, setPortfolioValue] = useState(0);
12   const [portfolio, setPortfolio] = useState([]);
13   const [stocks, setStocks] = useState([]);
14   const [transactions, setTransactions] = useState([]);
15   const [loading, setLoading] = useState(true);
16
17   useEffect(() => {
18     fetchDashboardData();
19   }, [user]);
20
21   const fetchDashboardData = async () => {
22     if (!user) return;
23
24     try {
25       setLoading(true);
26
27       // Fetch portfolio value
28       const valueResponse = await portfolioService.getPortfolioValue(user.username);
29       if (valueResponse.success) {
30         setPortfolioValue(valueResponse.data || 0);
31       }
32
33       // Fetch portfolio holdings
34       const portfolioResponse = await portfolioService.getUserPortfolio(user.username);
35       if (portfolioResponse.success) {
36         setPortfolio(portfolioResponse.data || []);
37       }
38
39       // Fetch stocks
40       const stocksResponse = await stockService.getAllStocks();
41       if (stocksResponse.success) {
42         setStocks(stocksResponse.data || []);
43       }
44
45       // Fetch recent transactions
46       const transactionsResponse = await portfolioService.getTransactionHistory(user.username);
47       if (transactionsResponse.success) {
48         setTransactions((transactionsResponse.data || []).slice(0, 5));
49       }
50     } catch (error) {
51       toast.error('Failed to load dashboard data');
52       console.error(error);
53     } finally {
54       setLoading(false);
55     }
56   };
57
58   const formatCurrency = (value) => {
59     return new Intl.NumberFormat('en-US', {
60       style: 'currency',
61       currency: 'USD',
62       minimumFractionDigits: 2,
63     }).format(value);
64   };
65
66   const calculateTotalValue = () => {
67     return (user?.balance || 0) + portfolioValue;
68   };
69
70   const calculateProfitLoss = () => {
71     const totalInvested = portfolio.reduce((sum, item) => {
```

```

72     return sum + (item.averagePrice * item.quantity);
73   }, 0);
74   return portfolioValue - totalInvested;
75 };
76
77 const getTopGainers = () => {
78   return stocks
79     .sort((a, b) => b.currentPrice - a.currentPrice)
80     .slice(0, 3);
81 };
82
83 if (loading) {
84   return (
85     <div className="dashboard-loading">
86       <div className="spinner"></div>
87       <p>Loading dashboard...</p>
88     </div>
89   );
90 }
91
92 return (
93   <div className="dashboard">
94     <div className="dashboard-header">
95       <h1>Welcome back, {user?.username}! █</h1>
96       <p>Here's your trading overview</p>
97     </div>
98
99     <div className="stats-grid">
100       <div className="stat-card glass-card">
101         <div className="stat-icon" style={{ background: 'linear-gradient(135deg,
#6366f1, #8b5cf6)' }}>
102           <Wallet size={24} />
103         </div>
104         <div className="stat-content">
105           <p className="stat-label">Cash Balance</p>
106           <h3 className="stat-value">{formatCurrency(user?.balance || 0)}</h3>
107         </div>
108       </div>
109
110       <div className="stat-card glass-card">
111         <div className="stat-icon" style={{ background: 'linear-gradient(135deg,
#10b981, #34d399)' }}>
112           <TrendingUp size={24} />
113         </div>
114         <div className="stat-content">
115           <p className="stat-label">Portfolio Value</p>
116           <h3 className="stat-value">{formatCurrency(portfolioValue)}</h3>
117         </div>
118       </div>
119
120       <div className="stat-card glass-card">
121         <div className="stat-icon" style={{ background: 'linear-gradient(135deg,
#f59e0b, #fbbf24)' }}>
122           <DollarSign size={24} />
123         </div>
124         <div className="stat-content">
125           <p className="stat-label">Total Value</p>
126           <h3 className="stat-value">{formatCurrency(calculateTotalValue())}</h3>
127         </div>
128       </div>
129
130       <div className="stat-card glass-card">
131         <div className="stat-icon" style={{ background: calculateProfitLoss() >= 0 ? '
linear-gradient(135deg, #10b981, #34d399)' : 'linear-gradient(135deg, #ef4444, #f87171)' }}>
132           {calculateProfitLoss() >= 0 ? <TrendingUp size={24} /> : <TrendingDown
size={24} />}
133         </div>
134         <div className="stat-content">
135           <p className="stat-label">Profit/Loss</p>
136           <h3 className={`stat-value ${calculateProfitLoss() >= 0 ? 'text-success' : '
text-danger'}`}>
137             {formatCurrency(calculateProfitLoss())}

```

```

138         </h3>
139     </div>
140 </div>
141 </div>
142
143     <div className="dashboard-content">
144         <div className="dashboard-section">
145             <h2>
146                 <Activity size={24} />
147                 Top Gainers
148             </h2>
149             <div className="top-stocks">
150                 {getTopGainers().map((stock) => (
151                     <div key={stock.id} className="stock-item glass-card">
152                         <div className="stock-info">
153                             <h4>{stock.symbol}</h4>
154                             <p>{stock.companyName}</p>
155                         </div>
156                         <div className="stock-price">
157                             <span className="price">{formatCurrency(stock.currentPrice)}</sp
158 an>
158                             </div>
159                         </div>
160                     )})
161                 </div>
162             </div>
163
164             <div className="dashboard-section">
165                 <h2>
166                     <TrendingUp size={24} />
167                     Recent Transactions
168                 </h2>
169                 <div className="transactions-list">
170                     {transactions.length > 0 ? (
171                         transactions.map((transaction) => (
172                             <div key={transaction.id} className="transaction-item glass-card">
173                                 <div className="transaction-type">
174                                     <span className={`badge ${transaction.transactionType === '
175 BUY' ? 'badge-success' : 'badge-danger'}`}>
176                                         {transaction.transactionType}
177                                     </span>
178                                 </div>
179                                 <div className="transaction-details">
180                                     <h4>{transaction.stockSymbol}</h4>
181                                     <p>{transaction.quantity} shares @
182                                     {formatCurrency(transaction.pricePerShare)}</p>
183                                     </div>
184                                     <div className="transaction-amount">
185                                         <span className={transaction.transactionType === 'BUY' ? '
186 text-danger' : 'text-success'}>
187                                             {transaction.transactionType === 'BUY' ? '-' : '
188 +'}{formatCurrency(transaction.totalAmount)}
189                                         </span>
190                                     </div>
191                                 </div>
192                             )})
193                         ) : (
194                             <p className="text-muted">No transactions yet. Start trading!</p>
195                         )}
196                     </div>
197                 </div>
198             </div>
199         </div>
200     );
201 };
202
203 export default Dashboard;
204

```

```

1 .navbar {
2     position: sticky;
3     top: 0;
4     z-index: var(--z-dropdown);
5     margin-bottom: var(--space-xl);
6     padding: var(--space-md) var(--space-xl);
7 }
8
9 .navbar-container {
10     display: flex;
11     align-items: center;
12     justify-content: space-between;
13     gap: var(--space-xl);
14 }
15
16 .navbar-brand {
17     display: flex;
18     align-items: center;
19     gap: var(--space-md);
20     font-size: var(--font-size-xl);
21     font-weight: 800;
22     color: var(--color-text-primary);
23     text-decoration: none;
24     transition: transform var(--transition-fast);
25 }
26
27 .navbar-brand:hover {
28     transform: scale(1.05);
29 }
30
31 .navbar-brand svg {
32     color: var(--color-primary);
33 }
34
35 .navbar-links {
36     display: flex;
37     align-items: center;
38     gap: var(--space-lg);
39     flex: 1;
40     justify-content: center;
41 }
42
43 .navbar-link {
44     color: var(--color-text-secondary);
45     text-decoration: none;
46     font-weight: 600;
47     font-size: var(--font-size-base);
48     padding: var(--space-sm) var(--space-md);
49     border-radius: var(--radius-md);
50     transition: all var(--transition-fast);
51     position: relative;
52 }
53
54 .navbar-link:hover {
55     color: var(--color-primary-light);
56     background: var(--color-surface-hover);
57 }
58
59 .navbar-link.active {
60     color: var(--color-primary);
61 }
62
63 .navbar-user {
64     display: flex;
65     align-items: center;
66     gap: var(--space-lg);
67 }
68
69 .navbar-balance {
70     display: flex;
71     align-items: center;

```

```

72     gap: var(--space-sm);
73     padding: var(--space-sm) var(--space-md);
74     background: var(--color-surface);
75     border-radius: var(--radius-md);
76     border: 1px solid var(--color-border);
77 }
78
79 .balance-amount {
80     font-weight: 700;
81     font-size: var(--font-size-lg);
82     color: var(--color-success);
83 }
84
85 .navbar-profile {
86     display: flex;
87     align-items: center;
88     gap: var(--space-sm);
89     padding: var(--space-sm) var(--space-md);
90     background: var(--color-surface);
91     border-radius: var(--radius-md);
92     border: 1px solid var(--color-border);
93     color: var(--color-text-secondary);
94 }
95
96 @media (max-width: 1024px) {
97     .navbar-links {
98         display: none;
99     }
100
101     .navbar-balance .balance-amount {
102         font-size: var(--font-size-base);
103     }
104 }
105
106 @media (max-width: 768px) {
107     .navbar {
108         padding: var(--space-md);
109     }
110
111     .navbar-container {
112         gap: var(--space-md);
113     }
114
115     .navbar-brand span {
116         display: none;
117     }
118
119     .navbar-user {
120         gap: var(--space-sm);
121     }
122
123     .navbar-profile span {
124         display: none;
125     }
126
127     .navbar-balance {
128         padding: var(--space-xs) var(--space-sm);
129     }
130 }

```

■ Navbar.jsx src/components/dashboard/Navbar.jsx

```
1 import { Link, useNavigate } from 'react-router-dom';
2 import { useAuth } from '../../context/AuthContext';
3 import { TrendingUp, LogOut, User, Wallet } from 'lucide-react';
4 import './Navbar.css';
5
6 const Navbar = () => {
7   const { user, logout } = useAuth();
8   const navigate = useNavigate();
9
10  const handleLogout = () => {
11    logout();
12    navigate('/login');
13  };
14
15  const formatBalance = (balance) => {
16    return new Intl.NumberFormat('en-US', {
17      style: 'currency',
18      currency: 'USD',
19      minimumFractionDigits: 2,
20    }).format(balance);
21  };
22
23  return (
24    <nav className="navbar glass-card">
25      <div className="navbar-container">
26        <Link to="/dashboard" className="navbar-brand">
27          <TrendingUp size={32} />
28          <span>MRU Trading</span>
29        </Link>
30
31        <div className="navbar-links">
32          <Link to="/dashboard" className="navbar-link">
33            Dashboard
34          </Link>
35          <Link to="/market" className="navbar-link">
36            Market
37          </Link>
38          <Link to="/portfolio" className="navbar-link">
39            Portfolio
40          </Link>
41          <Link to="/transactions" className="navbar-link">
42            Transactions
43          </Link>
44        </div>
45
46        <div className="navbar-user">
47          <div className="navbar-balance">
48            <Wallet size={20} />
49            <span className="balance-amount">{formatBalance(user?.balance || 0)}</span>
50          </div>
51
52          <div className="navbar-profile">
53            <User size={20} />
54            <span>{user?.username}</span>
55          </div>
56
57          <button onClick={handleLogout} className="btn btn-outline btn-sm">
58            <LogOut size={16} />
59            Logout
60          </button>
61        </div>
62      </div>
63    </nav>
64  );
65 };
66
67 export default Navbar;
68
```

```

1 .market {
2   padding: var(--space-xl);
3   max-width: 1400px;
4   margin: 0 auto;
5   animation: fadeIn 0.5s ease-out;
6 }
7
8 .market-loading {
9   display: flex;
10  flex-direction: column;
11  align-items: center;
12  justify-content: center;
13  min-height: 60vh;
14  gap: var(--space-lg);
15 }
16
17 .market-header {
18   display: flex;
19   align-items: center;
20   justify-content: space-between;
21   margin-bottom: var(--space-2xl);
22   gap: var(--space-xl);
23   flex-wrap: wrap;
24 }
25
26 .market-header h1 {
27   display: flex;
28   align-items: center;
29   gap: var(--space-md);
30   margin-bottom: var(--space-sm);
31 }
32
33 .market-header p {
34   color: var(--color-text-tertiary);
35   font-size: var(--font-size-lg);
36 }
37
38 .search-box {
39   display: flex;
40   align-items: center;
41   gap: var(--space-md);
42   background: var(--color-bg-tertiary);
43   border: 1px solid var(--color-border);
44   border-radius: var(--radius-md);
45   padding: var(--space-md);
46   min-width: 300px;
47 }
48
49 .search-input {
50   flex: 1;
51   background: transparent;
52   border: none;
53   color: var(--color-text-primary);
54   font-size: var(--font-size-base);
55   outline: none;
56 }
57
58 .search-input::placeholder {
59   color: var(--color-text-muted);
60 }
61
62 .stocks-grid {
63   display: grid;
64   grid-template-columns: repeat(auto-fill, minmax(300px, 1fr));
65   gap: var(--space-lg);
66 }
67
68 .stock-card {
69   padding: var(--space-xl);
70   transition: all var(--transition-base);
71 }

```

```

72
73 .stock-card:hover {
74     transform: translateY(-4px);
75     border-color: var(--color-primary);
76 }
77
78 .stock-card-header {
79     display: flex;
80     align-items: center;
81     justify-content: space-between;
82     margin-bottom: var(--space-lg);
83 }
84
85 .stock-symbol {
86     font-size: var(--font-size-2xl);
87     font-weight: 800;
88     color: var(--color-primary);
89 }
90
91 .stock-price {
92     font-size: var(--font-size-xl);
93     font-weight: 700;
94     color: var(--color-success);
95 }
96
97 .stock-card-body {
98     margin-bottom: var(--space-lg);
99 }
100
101 .stock-card-body h3 {
102     font-size: var(--font-size-lg);
103     margin-bottom: var(--space-sm);
104 }
105
106 .stock-card-footer {
107     display: flex;
108     gap: var(--space-sm);
109 }
110
111 .stock-card-footer .btn {
112     flex: 1;
113 }
114
115 .no-results {
116     text-align: center;
117     padding: var(--space-2xl);
118     color: var(--color-text-tertiary);
119 }
120
121 /* Modal Styles */
122 .modal-overlay {
123     position: fixed;
124     top: 0;
125     left: 0;
126     right: 0;
127     bottom: 0;
128     background: rgba(0, 0, 0, 0.7);
129     backdrop-filter: blur(4px);
130     display: flex;
131     align-items: center;
132     justify-content: center;
133     z-index: var(--z-modal);
134     padding: var(--space-lg);
135     animation: fadeIn 0.2s ease-out;
136 }
137
138 .modal {
139     width: 100%;
140     max-width: 500px;
141     max-height: 90vh;
142     overflow-y: auto;
143     animation: slideIn 0.3s ease-out;

```

```

144 }
145
146 .modal-header {
147     display: flex;
148     align-items: center;
149     justify-content: space-between;
150     padding: var(--space-xl);
151     border-bottom: 1px solid var(--color-divider);
152 }
153
154 .modal-header h2 {
155     font-size: var(--font-size-2xl);
156 }
157
158 .modal-close {
159     background: none;
160     border: none;
161     font-size: 2rem;
162     color: var(--color-text-secondary);
163     cursor: pointer;
164     width: 40px;
165     height: 40px;
166     display: flex;
167     align-items: center;
168     justify-content: center;
169     border-radius: var(--radius-md);
170     transition: all var(--transition-fast);
171 }
172
173 .modal-close:hover {
174     background: var(--color-surface-hover);
175     color: var(--color-text-primary);
176 }
177
178 .modal-body {
179     padding: var(--space-xl);
180 }
181
182 .modal-stock-info {
183     text-align: center;
184     margin-bottom: var(--space-xl);
185     padding: var(--space-lg);
186     background: var(--color-surface);
187     border-radius: var(--radius-md);
188 }
189
190 .modal-stock-info h3 {
191     margin-bottom: var(--space-sm);
192 }
193
194 .modal-price {
195     font-size: var(--font-size-2xl);
196     font-weight: 700;
197     color: var(--color-success);
198 }
199
200 .modal-summary {
201     margin-top: var(--space-xl);
202     padding: var(--space-lg);
203     background: var(--color-surface);
204     border-radius: var(--radius-md);
205 }
206
207 .summary-row {
208     display: flex;
209     justify-content: space-between;
210     padding: var(--space-sm) 0;
211     font-size: var(--font-size-base);
212 }
213
214 .summary-row.total {
215     border-top: 2px solid var(--color-border);

```

```

216     margin-top: var(--space-sm);
217     padding-top: var(--space-md);
218     font-size: var(--font-size-lg);
219     font-weight: 700;
220 }
221
222 .modal-footer {
223     display: flex;
224     gap: var(--space-md);
225     padding: var(--space-xl);
226     border-top: 1px solid var(--color-divider);
227 }
228
229 .modal-footer .btn {
230     flex: 1;
231 }
232
233 @media (max-width: 768px) {
234     .market {
235         padding: var(--space-md);
236     }
237
238     .market-header {
239         flex-direction: column;
240         align-items: flex-start;
241     }
242
243     .search-box {
244         width: 100%;
245         min-width: unset;
246     }
247
248     .stocks-grid {
249         grid-template-columns: 1fr;
250     }
251 }

```

■ Market.jsx src/components/market/Market.jsx

```
1 import { useState, useEffect } from 'react';
2 import { useAuth } from '../../../context/AuthContext';
3 import stockService from '../../../services/stockService';
4 import tradingService from '../../../services/tradingService';
5 import toast from 'react-hot-toast';
6 import { Search, TrendingUp, ShoppingCart, DollarSign } from 'lucide-react';
7 import './Market.css';
8
9 const Market = () => {
10   const { user, refreshUser } = useAuth();
11   const [stocks, setStocks] = useState([]);
12   const [filteredStocks, setFilteredStocks] = useState([]);
13   const [searchTerm, setSearchTerm] = useState('');
14   const [loading, setLoading] = useState(true);
15   const [selectedStock, setSelectedStock] = useState(null);
16   const [quantity, setQuantity] = useState(1);
17   const [buyingStock, setBuyingStock] = useState(false);
18
19   useEffect(() => {
20     fetchStocks();
21   }, []);
22
23   useEffect(() => {
24     filterStocks();
25   }, [searchTerm, stocks]);
26
27   const fetchStocks = async () => {
28     try {
29       setLoading(true);
30       const response = await stockService.getAllStocks();
31       if (response.success) {
32         setStocks(response.data || []);
33         setFilteredStocks(response.data || []);
34       }
35     } catch (error) {
36       toast.error('Failed to load stocks');
37       console.error(error);
38     } finally {
39       setLoading(false);
40     }
41   };
42
43   const filterStocks = () => {
44     if (!searchTerm) {
45       setFilteredStocks(stocks);
46       return;
47     }
48
49     const filtered = stocks.filter((stock) =>
50       stock.symbol.toLowerCase().includes(searchTerm.toLowerCase()) ||
51       stock.companyName.toLowerCase().includes(searchTerm.toLowerCase())
52     );
53     setFilteredStocks(filtered);
54   };
55
56   const handleBuyClick = (stock) => {
57     setSelectedStock(stock);
58     setQuantity(1);
59   };
60
61   const handleBuyStock = async () => {
62     if (!selectedStock || quantity < 1) return;
63
64     const totalCost = selectedStock.currentPrice * quantity;
65     if (totalCost > user.balance) {
66       toast.error('Insufficient balance!');
67       return;
68     }
69
70     try {
71       setBuyingStock(true);
```

```

72     const response = await tradingService.buyStock({
73         username: user.username,
74         stockSymbol: selectedStock.symbol,
75         quantity: quantity,
76     });
77
78     if (response.success) {
79         toast.success(`Successfully bought ${quantity} shares of ${selectedStock.symbol}!`);
80         setSelectedStock(null);
81         setQuantity(1);
82         await refreshUser();
83     } else {
84         toast.error(response.message || 'Failed to buy stock');
85     }
86 } catch (error) {
87     toast.error(error.message || 'Failed to buy stock');
88 } finally {
89     setBuyingStock(false);
90 }
91 };
92
93 const formatCurrency = (value) => {
94     return new Intl.NumberFormat('en-US', {
95         style: 'currency',
96         currency: 'USD',
97         minimumFractionDigits: 2,
98     }).format(value);
99 };
100
101 if (loading) {
102     return (
103         <div className="market-loading">
104             <div className="spinner"></div>
105             <p>Loading market data...</p>
106         </div>
107     );
108 }
109
110 return (
111     <div className="market">
112         <div className="market-header">
113             <div>
114                 <h1>
115                     <TrendingUp size={32} />
116                     Stock Market
117                 </h1>
118                 <p>Browse and trade available stocks</p>
119             </div>
120
121             <div className="search-box">
122                 <Search size={20} />
123                 <input
124                     type="text"
125                     placeholder="Search stocks by symbol or company..."
126                     value={searchTerm}
127                     onChange={(e) => setSearchTerm(e.target.value)}
128                     className="search-input"
129                 />
130             </div>
131         </div>
132
133         <div className="stocks-grid">
134             {filteredStocks.map((stock) => (
135                 <div key={stock.id} className="stock-card glass-card">
136                     <div className="stock-card-header">
137                         <div className="stock-symbol">{stock.symbol}</div>
138                         <div className="stock-price">{formatCurrency(stock.currentPrice)}</div>
139                     </div>
140
141                     <div className="stock-card-body">
142                         <h3>{stock.companyName}</h3>
143                         <p className="text-muted">Last updated: {new

```

```

Date(stock.lastUpdated).toLocaleString())</p>
144         </div>
145
146         <div className="stock-card-footer">
147             <button
148                 onClick={() => handleBuyClick(stock)}
149                 className="btn btn-success btn-sm"
150             >
151                 <ShoppingCart size={16} />
152                 Buy Stock
153             </button>
154         </div>
155     </div>
156     )}}
157 </div>
158
159     {filteredStocks.length === 0 && (
160         <div className="no-results">
161             <p>No stocks found matching "{searchTerm}"</p>
162         </div>
163     )}
164
165     {/* Buy Modal */}
166     {selectedStock && (
167         <div className="modal-overlay" onClick={() => setSelectedStock(null)}>
168             <div className="modal glass-card" onClick={(e) => e.stopPropagation()}>
169                 <div className="modal-header">
170                     <h2>Buy {selectedStock.symbol}</h2>
171                     <button onClick={() => setSelectedStock(null)}
172                         className="modal-close"></button>
173                 </div>
174
175                 <div className="modal-body">
176                     <div className="modal-stock-info">
177                         <h3>{selectedStock.companyName}</h3>
178                         <p className="modal-price">{formatCurrency(selectedStock.currentPrice)} per share</p>
179                     </div>
180
181                     <div className="form-group">
182                         <label className="form-label">Quantity</label>
183                         <input
184                             type="number"
185                             min="1"
186                             value={quantity}
187                             onChange={(e) => setQuantity(parseInt(e.target.value) || 1)}
188                             className="form-input"
189                         />
190                     </div>
191
192                     <div className="modal-summary">
193                         <div className="summary-row">
194                             <span>Price per share:</span>
195                             <span>{formatCurrency(selectedStock.currentPrice)}</span>
196                         </div>
197                         <div className="summary-row">
198                             <span>Quantity:</span>
199                             <span>{quantity}</span>
200                         </div>
201                         <div className="summary-row total">
202                             <span>Total Cost:</span>
203                             <span className="text-success">{formatCurrency(selectedStock.currentPrice * quantity)}</span>
204                         </div>
205                         <div className="summary-row">
206                             <span>Your Balance:</span>
207                             <span>{formatCurrency(user.balance)}</span>
208                         </div>
209                     </div>
210                 </div>
211                 <div className="modal-footer">

```

```

212             <button onClick={() => setSelectedStock(null)} className="btn
btn-outline">
213                 Cancel
214             </button>
215             <button
216                 onClick={handleBuyStock}
217                 className="btn btn-success"
218                 disabled={buyingStock || (selectedStock.currentPrice * quantity) >
user.balance}
219             >
220                 {buyingStock ? (
221                     <>
222                         <div className="spinner" style={{ width: '16px', height: '
16px', borderWidth: '2px' }}></div>
223                         Processing...
224                     </>
225                 ) : (
226                     <>
227                         <DollarSign size={16} />
228                         Confirm Purchase
229                     </>
230                 )}
231             </button>
232         </div>
233     </div>
234 </div>
235     )}
236 </div>
237 );
238 };
239
240 export default Market;
241

```

```

1 .portfolio {
2   padding: var(--space-xl);
3   max-width: 1400px;
4   margin: 0 auto;
5   animation: fadeIn 0.5s ease-out;
6 }
7
8 .portfolio-loading {
9   display: flex;
10  flex-direction: column;
11  align-items: center;
12  justify-content: center;
13  min-height: 60vh;
14  gap: var(--space-lg);
15 }
16
17 .portfolio-header {
18   display: flex;
19   align-items: center;
20   justify-content: space-between;
21   margin-bottom: var(--space-2xl);
22   gap: var(--space-xl);
23   flex-wrap: wrap;
24 }
25
26 .portfolio-header h1 {
27   display: flex;
28   align-items: center;
29   gap: var(--space-md);
30   margin-bottom: var(--space-sm);
31 }
32
33 .portfolio-header p {
34   color: var(--color-text-tertiary);
35   font-size: var(--font-size-lg);
36 }
37
38 .portfolio-value {
39   padding: var(--space-lg) var(--space-xl);
40   text-align: right;
41 }
42
43 .value-label {
44   font-size: var(--font-size-sm);
45   color: var(--color-text-tertiary);
46   margin-bottom: var(--space-xs);
47   text-transform: uppercase;
48   letter-spacing: 0.5px;
49 }
50
51 .value-amount {
52   font-size: var(--font-size-3xl);
53   font-weight: 800;
54   color: var(--color-success);
55 }
56
57 .holdings-grid {
58   display: grid;
59   grid-template-columns: repeat(auto-fill, minmax(350px, 1fr));
60   gap: var(--space-lg);
61 }
62
63 .holding-card {
64   padding: var(--space-xl);
65   transition: all var(--transition-base);
66 }
67
68 .holding-card:hover {
69   transform: translateY(-4px);
70 }
71

```

```

72 .holding-header {
73     display: flex;
74     align-items: center;
75     justify-content: space-between;
76     margin-bottom: var(--space-lg);
77     padding-bottom: var(--space-md);
78     border-bottom: 1px solid var(--color-divider);
79 }
80
81 .holding-symbol {
82     font-size: var(--font-size-2xl);
83     font-weight: 800;
84     color: var(--color-primary);
85 }
86
87 .holding-badge {
88     display: flex;
89     align-items: center;
90     gap: var(--space-xs);
91     padding: var(--space-xs) var(--space-md);
92     border-radius: var(--radius-full);
93     font-size: var(--font-size-sm);
94     font-weight: 700;
95 }
96
97 .holding-body {
98     margin-bottom: var(--space-lg);
99 }
100
101 .holding-row {
102     display: flex;
103     justify-content: space-between;
104     padding: var(--space-sm) 0;
105     font-size: var(--font-size-base);
106 }
107
108 .holding-row.profit-loss {
109     border-top: 2px solid var(--color-border);
110     margin-top: var(--space-sm);
111     padding-top: var(--space-md);
112     font-size: var(--font-size-lg);
113 }
114
115 .holding-value {
116     font-weight: 600;
117     color: var(--color-text-primary);
118 }
119
120 .holding-footer {
121     display: flex;
122     gap: var(--space-sm);
123 }
124
125 .holding-footer .btn {
126     flex: 1;
127 }
128
129 .empty-portfolio {
130     display: flex;
131     flex-direction: column;
132     align-items: center;
133     justify-content: center;
134     padding: var(--space-2xl);
135     text-align: center;
136     min-height: 400px;
137     gap: var(--space-lg);
138 }
139
140 .empty-portfolio svg {
141     color: var(--color-text-muted);
142     opacity: 0.5;
143 }

```

```
144
145 .empty-portfolio h3 {
146     font-size: var(--font-size-2xl);
147     color: var(--color-text-secondary);
148 }
149
150 .empty-portfolio p {
151     color: var(--color-text-tertiary);
152 }
153
154 @media (max-width: 768px) {
155     .portfolio {
156         padding: var(--space-md);
157     }
158
159     .portfolio-header {
160         flex-direction: column;
161         align-items: flex-start;
162     }
163
164     .portfolio-value {
165         width: 100%;
166         text-align: left;
167     }
168
169     .holdings-grid {
170         grid-template-columns: 1fr;
171     }
172 }
```

■ Portfolio.jsx src/components/portfolio/Portfolio.jsx

```
1 import { useState, useEffect } from 'react';
2 import { useAuth } from '../../../context/AuthContext';
3 import portfolioService from '../../../services/portfolioService';
4 import tradingService from '../../../services/tradingService';
5 import toast from 'react-hot-toast';
6 import { Briefcase, TrendingUp, TrendingDown, DollarSign } from 'lucide-react';
7 import './Portfolio.css';
8
9 const Portfolio = () => {
10   const { user, refreshUser } = useAuth();
11   const [portfolio, setPortfolio] = useState([]);
12   const [portfolioValue, setPortfolioValue] = useState(0);
13   const [loading, setLoading] = useState(true);
14   const [selectedStock, setSelectedStock] = useState(null);
15   const [sellQuantity, setSellQuantity] = useState(1);
16   const [sellingStock, setSellingStock] = useState(false);
17
18   useEffect(() => {
19     fetchPortfolio();
20   }, [user]);
21
22   const fetchPortfolio = async () => {
23     if (!user) return;
24
25     try {
26       setLoading(true);
27
28       const [portfolioResponse, valueResponse] = await Promise.all([
29         portfolioService.getUserPortfolio(user.username),
30         portfolioService.getPortfolioValue(user.username),
31       ]);
32
33       if (portfolioResponse.success) {
34         setPortfolio(portfolioResponse.data || []);
35       }
36
37       if (valueResponse.success) {
38         setPortfolioValue(valueResponse.data || 0);
39       }
40     } catch (error) {
41       toast.error('Failed to load portfolio');
42       console.error(error);
43     } finally {
44       setLoading(false);
45     }
46   };
47
48   const handleSellClick = (holding) => {
49     setSelectedStock(holding);
50     setSellQuantity(1);
51   };
52
53   const handleSellStock = async () => {
54     if (!selectedStock || sellQuantity < 1 || sellQuantity > selectedStock.quantity) return;
55
56     try {
57       setSellingStock(true);
58       const response = await tradingService.sellStock({
59         username: user.username,
60         stockSymbol: selectedStock.stockSymbol,
61         quantity: sellQuantity,
62       });
63
64       if (response.success) {
65         toast.success(`Successfully sold ${sellQuantity} shares of
66 ${selectedStock.stockSymbol}!`);
67         setSelectedStock(null);
68         setSellQuantity(1);
69         await refreshUser();
70         await fetchPortfolio();
71       } else {
```

```

71         toast.error(response.message || 'Failed to sell stock');
72     }
73     } catch (error) {
74         toast.error(error.message || 'Failed to sell stock');
75     } finally {
76         setSellingStock(false);
77     }
78 };
79
80 const formatCurrency = (value) => {
81     return new Intl.NumberFormat('en-US', {
82         style: 'currency',
83         currency: 'USD',
84         minimumFractionDigits: 2,
85     }).format(value);
86 };
87
88 const calculateProfitLoss = (holding) => {
89     return (holding.currentPrice - holding.averagePrice) * holding.quantity;
90 };
91
92 const calculateProfitLossPercentage = (holding) => {
93     return ((holding.currentPrice - holding.averagePrice) / holding.averagePrice) * 100;
94 };
95
96 if (loading) {
97     return (
98         <div className="portfolio-loading">
99             <div className="spinner"></div>
100             <p>Loading portfolio...</p>
101         </div>
102     );
103 }
104
105 return (
106     <div className="portfolio">
107         <div className="portfolio-header">
108             <div>
109                 <h1>
110                     <Briefcase size={32} />
111                     My Portfolio
112                 </h1>
113                 <p>Manage your stock holdings</p>
114             </div>
115
116             <div className="portfolio-value glass-card">
117                 <p className="value-label">Total Portfolio Value</p>
118                 <h2 className="value-amount">{formatCurrency(portfolioValue)}</h2>
119             </div>
120         </div>
121
122         {portfolio.length > 0 ? (
123             <div className="holdings-grid">
124                 {portfolio.map((holding) => {
125                     const profitLoss = calculateProfitLoss(holding);
126                     const profitLossPercent = calculateProfitLossPercentage(holding);
127                     const isProfit = profitLoss >= 0;
128
129                     return (
130                         <div key={holding.id} className="holding-card glass-card">
131                             <div className="holding-header">
132                                 <div className="holding-symbol">{holding.stockSymbol}</div>
133                                 <div className={`holding-badge ${isProfit ? 'badge-success' : '
badge-danger'} ``}>
134                                     {isProfit ? <TrendingUp size={16} /> : <TrendingDown
size={16} />}
135                                 <div>
136                                     {profitLossPercent.toFixed(2)}%
137                                 </div>
138                             </div>
139
140                             <div className="holding-body">
                                <div className="holding-row">

```

```

141         <span className="text-muted">Quantity:</span>
142         <span className="holding-value">{holding.quantity}
shares</span>
143     </div>
144
145     <div className="holding-row">
146         <span className="text-muted">Avg. Price:</span>
147         <span className="holding-value">{formatCurrency(holding.aver
agePrice)}</span>
148     </div>
149
150     <div className="holding-row">
151         <span className="text-muted">Current Price:</span>
152         <span className="holding-value">{formatCurrency(holding.curr
entPrice)}</span>
153     </div>
154
155     <div className="holding-row">
156         <span className="text-muted">Total Value:</span>
157         <span className="holding-value">{formatCurrency(holding.curr
entValue)}</span>
158     </div>
159
160     <div className="holding-row profit-loss">
161         <span className="text-muted">Profit/Loss:</span>
162         <span className={isProfit ? 'text-success' : 'text-danger'}>
163             {isProfit ? '+' : ''}{formatCurrency(profitLoss)}
164         </span>
165     </div>
166 </div>
167
168     <div className="holding-footer">
169         <button
170             onClick={() => handleSellClick(holding)}
171             className="btn btn-danger btn-sm"
172             >
173             <DollarSign size={16} />
174             Sell Stock
175         </button>
176     </div>
177 </div>
178 );
179 }}}
180 </div>
181 ) : (
182     <div className="empty-portfolio glass-card">
183         <Briefcase size={64} />
184         <h3>Your portfolio is empty</h3>
185         <p>Start trading to build your portfolio!</p>
186     </div>
187 )}
188
189 /* Sell Modal */
190 {selectedStock && (
191     <div className="modal-overlay" onClick={() => setSelectedStock(null)}>
192         <div className="modal glass-card" onClick={(e) => e.stopPropagation()}>
193             <div className="modal-header">
194                 <h2>Sell {selectedStock.stockSymbol}</h2>
195                 <button onClick={() => setSelectedStock(null)}
className="modal-close">x</button>
196             </div>
197
198             <div className="modal-body">
199                 <div className="modal-stock-info">
200                     <h3>{selectedStock.stockSymbol}</h3>
201                     <p className="modal-price">{formatCurrency(selectedStock.currentPric
e)} per share</p>
202                 </div>
203
204                 <div className="form-group">
205                     <label className="form-label">Quantity (Max:
{selectedStock.quantity}</label>

```

```

206         <input
207             type="number"
208             min="1"
209             max={selectedStock.quantity}
210             value={sellQuantity}
211             onChange={(e) => setSellQuantity(parseInt(e.target.value) || 1)}
212             className="form-input"
213         />
214     </div>
215
216     <div className="modal-summary">
217         <div className="summary-row">
218             <span>Price per share:</span>
219             <span>{formatCurrency(selectedStock.currentPrice)}</span>
220         </div>
221         <div className="summary-row">
222             <span>Quantity:</span>
223             <span>{sellQuantity}</span>
224         </div>
225         <div className="summary-row total">
226             <span>Total Amount:</span>
227             <span className="text-success">{formatCurrency(selectedStock.cur
228 rentPrice * sellQuantity)}</span>
229         </div>
230     </div>
231
232     <div className="modal-footer">
233         <button onClick={() => setSelectedStock(null)} className="btn
234 btn-outline">
235             Cancel
236         </button>
237         <button
238             onClick={handleSellStock}
239             className="btn btn-danger"
240             disabled={sellingStock || sellQuantity > selectedStock.quantity}
241             >
242             {sellingStock ? (
243                 <div className="spinner" style={{ width: '16px', height: '
244 16px', borderWidth: '2px' }}></div>
245                 Processing...
246             ) : (
247                 <div>
248                     <DollarSign size={16} />
249                     Confirm Sale
250                 </div>
251             )}
252         </button>
253     </div>
254 </div>
255 </div>
256 })
257 </div>
258 );
259 };
260
261 export default Portfolio;
262

```

```

1 .transactions {
2   padding: var(--space-xl);
3   max-width: 1200px;
4   margin: 0 auto;
5   animation: fadeIn 0.5s ease-out;
6 }
7
8 .transactions-loading {
9   display: flex;
10  flex-direction: column;
11  align-items: center;
12  justify-content: center;
13  min-height: 60vh;
14  gap: var(--space-lg);
15 }
16
17 .transactions-header {
18   display: flex;
19   align-items: center;
20   justify-content: space-between;
21   margin-bottom: var(--space-2xl);
22   gap: var(--space-xl);
23   flex-wrap: wrap;
24 }
25
26 .transactions-header h1 {
27   display: flex;
28   align-items: center;
29   gap: var(--space-md);
30   margin-bottom: var(--space-sm);
31 }
32
33 .transactions-header p {
34   color: var(--color-text-tertiary);
35   font-size: var(--font-size-lg);
36 }
37
38 .filter-buttons {
39   display: flex;
40   gap: var(--space-sm);
41 }
42
43 .transactions-list {
44   display: flex;
45   flex-direction: column;
46   gap: var(--space-md);
47 }
48
49 .transaction-card {
50   padding: var(--space-lg);
51   transition: all var(--transition-fast);
52 }
53
54 .transaction-card:hover {
55   transform: translateX(4px);
56   border-color: var(--color-primary);
57 }
58
59 .transaction-badge-wrapper {
60   margin-bottom: var(--space-md);
61 }
62
63 .transaction-content {
64   display: flex;
65   align-items: center;
66   justify-content: space-between;
67   gap: var(--space-lg);
68 }
69
70 .transaction-main {
71   flex: 1;

```

```

72 }
73
74 .transaction-main h3 {
75     font-size: var(--font-size-xl);
76     margin-bottom: var(--space-xs);
77     color: var(--color-primary);
78 }
79
80 .transaction-details {
81     font-size: var(--font-size-base);
82     color: var(--color-text-secondary);
83     margin-bottom: var(--space-xs);
84 }
85
86 .transaction-date {
87     font-size: var(--font-size-sm);
88     color: var(--color-text-muted);
89 }
90
91 .transaction-amount-wrapper {
92     display: flex;
93     flex-direction: column;
94     align-items: flex-end;
95     gap: var(--space-sm);
96 }
97
98 .transaction-amount {
99     font-size: var(--font-size-2xl);
100     font-weight: 800;
101 }
102
103 .transaction-status {
104     font-size: var(--font-size-xs);
105 }
106
107 .empty-transactions {
108     display: flex;
109     flex-direction: column;
110     align-items: center;
111     justify-content: center;
112     padding: var(--space-2xl);
113     text-align: center;
114     min-height: 400px;
115     gap: var(--space-lg);
116 }
117
118 .empty-transactions svg {
119     color: var(--color-text-muted);
120     opacity: 0.5;
121 }
122
123 .empty-transactions h3 {
124     font-size: var(--font-size-2xl);
125     color: var(--color-text-secondary);
126 }
127
128 .empty-transactions p {
129     color: var(--color-text-tertiary);
130 }
131
132 @media (max-width: 768px) {
133     .transactions {
134         padding: var(--space-md);
135     }
136
137     .transactions-header {
138         flex-direction: column;
139         align-items: flex-start;
140     }
141
142     .filter-buttons {
143         width: 100%;

```

```
144     justify-content: stretch;
145 }
146
147 .filter-buttons .btn {
148     flex: 1;
149 }
150
151 .transaction-content {
152     flex-direction: column;
153     align-items: flex-start;
154 }
155
156 .transaction-amount-wrapper {
157     width: 100%;
158     flex-direction: row;
159     justify-content: space-between;
160     align-items: center;
161 }
162 }
```

■ Transactions.jsx src/components/transactions/Transactions.jsx

```
1 import { useState, useEffect } from 'react';
2 import { useAuth } from '../../../context/AuthContext';
3 import portfolioService from '../../../services/portfolioService';
4 import toast from 'react-hot-toast';
5 import { Receipt, TrendingUp, TrendingDown, Filter } from 'lucide-react';
6 import './Transactions.css';
7
8 const Transactions = () => {
9   const { user } = useAuth();
10   const [transactions, setTransactions] = useState([]);
11   const [filteredTransactions, setFilteredTransactions] = useState([]);
12   const [loading, setLoading] = useState(true);
13   const [filter, setFilter] = useState('ALL'); // ALL, BUY, SELL
14
15   useEffect(() => {
16     fetchTransactions();
17   }, [user]);
18
19   useEffect(() => {
20     filterTransactions();
21   }, [filter, transactions]);
22
23   const fetchTransactions = async () => {
24     if (!user) return;
25
26     try {
27       setLoading(true);
28       const response = await portfolioService.getTransactionHistory(user.username);
29       if (response.success) {
30         setTransactions(response.data || []);
31         setFilteredTransactions(response.data || []);
32       }
33     } catch (error) {
34       toast.error('Failed to load transactions');
35       console.error(error);
36     } finally {
37       setLoading(false);
38     }
39   };
40
41   const filterTransactions = () => {
42     if (filter === 'ALL') {
43       setFilteredTransactions(transactions);
44     } else {
45       setFilteredTransactions(transactions.filter(t => t.transactionType === filter));
46     }
47   };
48
49   const formatCurrency = (value) => {
50     return new Intl.NumberFormat('en-US', {
51       style: 'currency',
52       currency: 'USD',
53       minimumFractionDigits: 2,
54     }).format(value);
55   };
56
57   const formatDate = (dateString) => {
58     const date = new Date(dateString);
59     return date.toLocaleString('en-US', {
60       year: 'numeric',
61       month: 'short',
62       day: 'numeric',
63       hour: '2-digit',
64       minute: '2-digit',
65     });
66   };
67
68   if (loading) {
69     return (
70       <div className="transactions-loading">
71         <div className="spinner"></div>
```

```

72         <p>Loading transactions...</p>
73     </div>
74     );
75 }
76
77 return (
78     <div className="transactions">
79         <div className="transactions-header">
80             <div>
81                 <h1>
82                     <Receipt size={32} />
83                     Transaction History
84                 </h1>
85                 <p>View all your trading activities</p>
86             </div>
87
88             <div className="filter-buttons">
89                 <button
90                     className={`btn btn-sm ${filter === 'ALL' ? 'btn-primary' : 'btn-outline'}}`
91                     onClick={() => setFilter('ALL')}
92                 >
93                     All
94                 </button>
95                 <button
96                     className={`btn btn-sm ${filter === 'BUY' ? 'btn-success' : 'btn-outline'}}`
97                     onClick={() => setFilter('BUY')}
98                 >
99                     <TrendingUp size={16} />
100                     Buy
101                 </button>
102                 <button
103                     className={`btn btn-sm ${filter === 'SELL' ? 'btn-danger' : 'btn-outline'}}`
104                     onClick={() => setFilter('SELL')}
105                 >
106                     <TrendingDown size={16} />
107                     Sell
108                 </button>
109             </div>
110         </div>
111
112         {filteredTransactions.length > 0 ? (
113             <div className="transactions-list">
114                 {filteredTransactions.map((transaction) => (
115                     <div key={transaction.id} className="transaction-card glass-card">
116                         <div className="transaction-badge-wrapper">
117                             <span className={`badge ${transaction.transactionType === 'BUY' ? '
badge-success' : 'badge-danger'}}`>
118                                 {transaction.transactionType === 'BUY' ? <TrendingUp size={14}
/> : <TrendingDown size={14} />}
119                                 {transaction.transactionType}
120                             </span>
121                         </div>
122
123                         <div className="transaction-content">
124                             <div className="transaction-main">
125                                 <h3>{transaction.stockSymbol}</h3>
126                                 <p className="transaction-details">
127                                     {transaction.quantity} shares @
128                                     {formatCurrency(transaction.pricePerShare)}
129                                     <p className="transaction-date">{formatDate(transaction.transact
ionDate)}</p>
130                                 </div>
131
132                                 <div className="transaction-amount-wrapper">
133                                     <div className={`transaction-amount
${transaction.transactionType === 'BUY' ? 'text-danger' : 'text-success'}}`>
134                                         {transaction.transactionType === 'BUY' ? '-' : '+'}
135                                         {formatCurrency(transaction.totalAmount)}
136                                     </div>
137                                     <span className={`transaction-status badge ${transaction.status
=== 'SUCCESS' ? 'badge-success' : 'badge-danger'}}`>

```

```

138             {transaction.status}
139         </span>
140     </div>
141 </div>
142 </div>
143     )}}
144 </div>
145 ) : (
146     <div className="empty-transactions glass-card">
147         <Receipt size={64} />
148         <h3>No transactions found</h3>
149         <p>
150             {filter === 'ALL'
151               ? 'Start trading to see your transaction history!'
152               : `No ${filter.toLowerCase()} transactions found.`}
153         </p>
154     </div>
155   )}
156 </div>
157 );
158 };
159
160 export default Transactions;
161

```

AuthContext.jsx src/context/AuthContext.jsx

```
1 import { createContext, useContext, useState, useEffect } from 'react';
2 import authService from '../services/authService';
3
4 const AuthContext = createContext(null);
5
6 export const useAuth = () => {
7   const context = useContext(AuthContext);
8   if (!context) {
9     throw new Error('useAuth must be used within an AuthProvider');
10  }
11  return context;
12 };
13
14 export const AuthProvider = ({ children }) => {
15   const [user, setUser] = useState(null);
16   const [loading, setLoading] = useState(true);
17
18   useEffect(() => {
19     // Check if user is logged in on mount
20     const currentUser = authService.getCurrentUser();
21     if (currentUser) {
22       setUser(currentUser);
23     }
24     setLoading(false);
25   }, []);
26
27   const login = async (credentials) => {
28     const response = await authService.login(credentials);
29     if (response.success && response.data) {
30       setUser(response.data);
31     }
32     return response;
33   };
34
35   const register = async (userData) => {
36     const response = await authService.register(userData);
37     return response;
38   };
39
40   const logout = () => {
41     authService.logout();
42     setUser(null);
43   };
44
45   const refreshUser = async () => {
46     if (user && user.username) {
47       try {
48         const response = await authService.getUserByUsername(user.username);
49         if (response.success && response.data) {
50           setUser(response.data);
51         }
52       } catch (error) {
53         console.error('Failed to refresh user:', error);
54       }
55     }
56   };
57
58   const value = {
59     user,
60     login,
61     register,
62     logout,
63     refreshUser,
64     isAuthenticated: !!user,
65     loading,
66   };
67
68   return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
69 };
70
71 export default AuthContext;
```

api.js `src/services/api.js`

```
1 import axios from 'axios';
2
3 // Create axios instance with base configuration
4 const api = axios.create({
5   baseURL: 'http://localhost:8080/api',
6   headers: {
7     'Content-Type': 'application/json',
8   },
9   timeout: 10000,
10 });
11
12 // Request interceptor
13 api.interceptors.request.use(
14   (config) => {
15     // You can add auth tokens here if needed
16     // const token = localStorage.getItem('token');
17     // if (token) {
18     //   config.headers.Authorization = `Bearer ${token}`;
19     // }
20     return config;
21   },
22   (error) => {
23     return Promise.reject(error);
24   }
25 );
26
27 // Response interceptor
28 api.interceptors.response.use(
29   (response) => {
30     return response.data;
31   },
32   (error) => {
33     // Handle errors globally
34     const errorMessage = error.response?.data?.message || error.message || 'An error occurred';
35     console.error('API Error:', errorMessage);
36     return Promise.reject(error);
37   }
38 );
39
40 export default api;
41
```

```
1 import api from './api';
2
3 const authService = {
4   // Register new user
5   register: async (userData) => {
6     try {
7       const response = await api.post('/users/register', userData);
8       return response;
9     } catch (error) {
10       throw error.response?.data || error;
11     }
12   },
13
14   // Login user
15   login: async (credentials) => {
16     try {
17       const response = await api.post('/users/login', credentials);
18       if (response.success && response.data) {
19         // Store user data in localStorage
20         localStorage.setItem('user', JSON.stringify(response.data));
21       }
22       return response;
23     } catch (error) {
24       throw error.response?.data || error;
25     }
26   },
27
28   // Get user by username
29   getUserByUsername: async (username) => {
30     try {
31       const response = await api.get(`/users/username/${username}`);
32       if (response.success && response.data) {
33         // Update user data in localStorage
34         localStorage.setItem('user', JSON.stringify(response.data));
35       }
36       return response;
37     } catch (error) {
38       throw error.response?.data || error;
39     }
40   },
41
42   // Logout user
43   logout: () => {
44     localStorage.removeItem('user');
45   },
46
47   // Get current user from localStorage
48   getCurrentUser: () => {
49     const userStr = localStorage.getItem('user');
50     if (userStr) {
51       try {
52         return JSON.parse(userStr);
53       } catch (error) {
54         return null;
55       }
56     }
57     return null;
58   },
59
60   // Check if user is authenticated
61   isAuthenticated: () => {
62     return !!authService.getCurrentUser();
63   },
64 };
65
66 export default authService;
67
```

■ portfolioService.js src/services/portfolioService.js

```
1 import api from './api';
2
3 const portfolioService = {
4   // Get user portfolio
5   getUserPortfolio: async (username) => {
6     try {
7       const response = await api.get(`/portfolio/${username}`);
8       return response;
9     } catch (error) {
10       throw error.response?.data || error;
11     }
12   },
13
14   // Get portfolio value
15   getPortfolioValue: async (username) => {
16     try {
17       const response = await api.get(`/portfolio/${username}/value`);
18       return response;
19     } catch (error) {
20       throw error.response?.data || error;
21     }
22   },
23
24   // Get transaction history
25   getTransactionHistory: async (username) => {
26     try {
27       const response = await api.get(`/portfolio/${username}/transactions`);
28       return response;
29     } catch (error) {
30       throw error.response?.data || error;
31     }
32   },
33 };
34
35 export default portfolioService;
36
```

■ stockService.js src/services/stockService.js

```
1 import api from './api';
2
3 const stockService = {
4   // Get all stocks
5   getAllStocks: async () => {
6     try {
7       const response = await api.get('/stocks');
8       return response;
9     } catch (error) {
10       throw error.response?.data || error;
11     }
12   },
13
14   // Get stock by symbol
15   getStockBySymbol: async (symbol) => {
16     try {
17       const response = await api.get(`/stocks/${symbol}`);
18       return response;
19     } catch (error) {
20       throw error.response?.data || error;
21     }
22   },
23 };
24
25 export default stockService;
26
```

```

1 import api from './api';
2
3 const tradingService = {
4   // Buy stock
5   buyStock: async (buyRequest) => {
6     try {
7       const response = await api.post('/trading/buy', buyRequest);
8       return response;
9     } catch (error) {
10       throw error.response?.data || error;
11     }
12   },
13
14   // Sell stock
15   sellStock: async (sellRequest) => {
16     try {
17       const response = await api.post('/trading/sell', sellRequest);
18       return response;
19     } catch (error) {
20       throw error.response?.data || error;
21     }
22   },
23 };
24
25 export default tradingService;
26

```

```

1 /* ===== CSS RESET & BASE STYLES ===== */
2 * {
3   margin: 0;
4   padding: 0;
5   box-sizing: border-box;
6 }
7
8 /* ===== CSS VARIABLES (DESIGN TOKENS) ===== */
9 :root {
10   /* Colors - Dark Theme */
11   --color-bg-primary: #0a0e27;
12   --color-bg-secondary: #1a1f3a;
13   --color-bg-tertiary: #252b48;
14   --color-surface: rgba(255, 255, 255, 0.05);
15   --color-surface-hover: rgba(255, 255, 255, 0.08);
16
17   /* Brand Colors */
18   --color-primary: #6366f1;
19   --color-primary-dark: #4f46e5;
20   --color-primary-light: #818cf8;
21   --color-accent: #8b5cf6;
22   --color-accent-light: #a78bfa;
23
24   /* Semantic Colors */
25   --color-success: #10b981;
26   --color-success-light: #34d399;
27   --color-danger: #ef4444;
28   --color-danger-light: #f87171;
29   --color-warning: #f59e0b;
30   --color-info: #3b82f6;
31
32   /* Text Colors */
33   --color-text-primary: #f8fafc;
34   --color-text-secondary: #cbd5e1;
35   --color-text-tertiary: #94a3b8;
36   --color-text-muted: #64748b;
37
38   /* Border & Divider */
39   --color-border: rgba(255, 255, 255, 0.1);
40   --color-divider: rgba(255, 255, 255, 0.05);
41
42   /* Shadows */
43   --shadow-sm: 0 1px 2px 0 rgba(0, 0, 0, 0.3);
44   --shadow-md: 0 4px 6px -1px rgba(0, 0, 0, 0.4);
45   --shadow-lg: 0 10px 15px -3px rgba(0, 0, 0, 0.5);
46   --shadow-xl: 0 20px 25px -5px rgba(0, 0, 0, 0.6);
47   --shadow-glow: 0 0 20px rgba(99, 102, 241, 0.3);
48
49   /* Spacing */
50   --space-xs: 0.25rem;
51   --space-sm: 0.5rem;
52   --space-md: 1rem;
53   --space-lg: 1.5rem;
54   --space-xl: 2rem;
55   --space-2xl: 3rem;
56
57   /* Border Radius */
58   --radius-sm: 0.375rem;
59   --radius-md: 0.5rem;
60   --radius-lg: 0.75rem;
61   --radius-xl: 1rem;
62   --radius-full: 9999px;
63
64   /* Typography */
65   --font-family: 'Inter', -apple-system, BlinkMacSystemFont, 'Segoe UI', sans-serif;
66   --font-size-xs: 0.75rem;
67   --font-size-sm: 0.875rem;
68   --font-size-base: 1rem;
69   --font-size-lg: 1.125rem;
70   --font-size-xl: 1.25rem;
71   --font-size-2xl: 1.5rem;

```

```

72 --font-size-3xl: 1.875rem;
73 --font-size-4xl: 2.25rem;
74
75 /* Transitions */
76 --transition-fast: 150ms cubic-bezier(0.4, 0, 0.2, 1);
77 --transition-base: 250ms cubic-bezier(0.4, 0, 0.2, 1);
78 --transition-slow: 350ms cubic-bezier(0.4, 0, 0.2, 1);
79
80 /* Z-index */
81 --z-dropdown: 1000;
82 --z-modal: 1050;
83 --z-tooltip: 1100;
84 }
85
86 /* ===== BASE STYLES ===== */
87 body {
88   font-family: var(--font-family);
89   background: var(--color-bg-primary);
90   color: var(--color-text-primary);
91   line-height: 1.6;
92   -webkit-font-smoothing: antialiased;
93   -moz-osx-font-smoothing: grayscale;
94   overflow-x: hidden;
95 }
96
97 #root {
98   min-height: 100vh;
99 }
100
101 /* ===== TYPOGRAPHY ===== */
102 h1, h2, h3, h4, h5, h6 {
103   font-weight: 700;
104   line-height: 1.2;
105   color: var(--color-text-primary);
106 }
107
108 h1 { font-size: var(--font-size-4xl); }
109 h2 { font-size: var(--font-size-3xl); }
110 h3 { font-size: var(--font-size-2xl); }
111 h4 { font-size: var(--font-size-xl); }
112 h5 { font-size: var(--font-size-lg); }
113 h6 { font-size: var(--font-size-base); }
114
115 p {
116   color: var(--color-text-secondary);
117 }
118
119 /* ===== GLASSMORPHISM CARD ===== */
120 .glass-card {
121   background: rgba(26, 31, 58, 0.6);
122   backdrop-filter: blur(20px);
123   -webkit-backdrop-filter: blur(20px);
124   border: 1px solid var(--color-border);
125   border-radius: var(--radius-lg);
126   box-shadow: var(--shadow-lg);
127   transition: all var(--transition-base);
128 }
129
130 .glass-card:hover {
131   background: rgba(26, 31, 58, 0.7);
132   border-color: rgba(255, 255, 255, 0.15);
133   transform: translateY(-2px);
134   box-shadow: var(--shadow-xl);
135 }
136
137 /* ===== BUTTONS ===== */
138 .btn {
139   display: inline-flex;
140   align-items: center;
141   justify-content: center;
142   gap: var(--space-sm);
143   padding: var(--space-sm) var(--space-lg);

```

```

144 font-size: var(--font-size-sm);
145 font-weight: 600;
146 border-radius: var(--radius-md);
147 border: none;
148 cursor: pointer;
149 transition: all var(--transition-fast);
150 text-decoration: none;
151 white-space: nowrap;
152 }
153
154 .btn-primary {
155   background: linear-gradient(135deg, var(--color-primary), var(--color-accent));
156   color: white;
157   box-shadow: 0 4px 12px rgba(99, 102, 241, 0.3);
158 }
159
160 .btn-primary:hover {
161   transform: translateY(-2px);
162   box-shadow: 0 6px 20px rgba(99, 102, 241, 0.4);
163 }
164
165 .btn-primary:active {
166   transform: translateY(0);
167 }
168
169 .btn-success {
170   background: var(--color-success);
171   color: white;
172 }
173
174 .btn-success:hover {
175   background: var(--color-success-light);
176   transform: translateY(-2px);
177 }
178
179 .btn-danger {
180   background: var(--color-danger);
181   color: white;
182 }
183
184 .btn-danger:hover {
185   background: var(--color-danger-light);
186   transform: translateY(-2px);
187 }
188
189 .btn-outline {
190   background: transparent;
191   border: 2px solid var(--color-border);
192   color: var(--color-text-primary);
193 }
194
195 .btn-outline:hover {
196   background: var(--color-surface-hover);
197   border-color: var(--color-primary);
198 }
199
200 .btn-sm {
201   padding: var(--space-xs) var(--space-md);
202   font-size: var(--font-size-xs);
203 }
204
205 .btn-lg {
206   padding: var(--space-md) var(--space-xl);
207   font-size: var(--font-size-base);
208 }
209
210 .btn:disabled {
211   opacity: 0.5;
212   cursor: not-allowed;
213   transform: none !important;
214 }
215

```

```

216 /* ===== FORM ELEMENTS ===== */
217 .form-group {
218     margin-bottom: var(--space-lg);
219 }
220
221 .form-label {
222     display: block;
223     margin-bottom: var(--space-sm);
224     font-size: var(--font-size-sm);
225     font-weight: 600;
226     color: var(--color-text-secondary);
227 }
228
229 .form-input {
230     width: 100%;
231     padding: var(--space-md);
232     background: var(--color-bg-tertiary);
233     border: 1px solid var(--color-border);
234     border-radius: var(--radius-md);
235     color: var(--color-text-primary);
236     font-size: var(--font-size-base);
237     font-family: var(--font-family);
238     transition: all var(--transition-fast);
239 }
240
241 .form-input:focus {
242     outline: none;
243     border-color: var(--color-primary);
244     box-shadow: 0 0 0 3px rgba(99, 102, 241, 0.1);
245 }
246
247 .form-input::placeholder {
248     color: var(--color-text-muted);
249 }
250
251 .form-error {
252     margin-top: var(--space-xs);
253     font-size: var(--font-size-sm);
254     color: var(--color-danger);
255 }
256
257 /* ===== BADGES ===== */
258 .badge {
259     display: inline-flex;
260     align-items: center;
261     padding: var(--space-xs) var(--space-sm);
262     font-size: var(--font-size-xs);
263     font-weight: 600;
264     border-radius: var(--radius-full);
265     text-transform: uppercase;
266     letter-spacing: 0.5px;
267 }
268
269 .badge-success {
270     background: rgba(16, 185, 129, 0.2);
271     color: var(--color-success-light);
272 }
273
274 .badge-danger {
275     background: rgba(239, 68, 68, 0.2);
276     color: var(--color-danger-light);
277 }
278
279 .badge-info {
280     background: rgba(59, 130, 246, 0.2);
281     color: var(--color-info);
282 }
283
284 .badge-warning {
285     background: rgba(245, 158, 11, 0.2);
286     color: var(--color-warning);
287 }

```

```

288
289 /* ===== ANIMATIONS ===== */
290 @keyframes fadeIn {
291   from {
292     opacity: 0;
293     transform: translateY(10px);
294   }
295   to {
296     opacity: 1;
297     transform: translateY(0);
298   }
299 }
300
301 @keyframes slideIn {
302   from {
303     transform: translateX(-100%);
304   }
305   to {
306     transform: translateX(0);
307   }
308 }
309
310 @keyframes pulse {
311   0%, 100% {
312     opacity: 1;
313   }
314   50% {
315     opacity: 0.5;
316   }
317 }
318
319 @keyframes shimmer {
320   0% {
321     background-position: -1000px 0;
322   }
323   100% {
324     background-position: 1000px 0;
325   }
326 }
327
328 .fade-in {
329   animation: fadeIn var(--transition-base) ease-out;
330 }
331
332 .slide-in {
333   animation: slideIn var(--transition-base) ease-out;
334 }
335
336 .pulse {
337   animation: pulse 2s cubic-bezier(0.4, 0, 0.6, 1) infinite;
338 }
339
340 /* ===== LOADING STATES ===== */
341 .skeleton {
342   background: linear-gradient(
343     90deg,
344     var(--color-bg-tertiary) 0%,
345     var(--color-surface) 50%,
346     var(--color-bg-tertiary) 100%
347   );
348   background-size: 200% 100%;
349   animation: shimmer 1.5s infinite;
350   border-radius: var(--radius-md);
351 }
352
353 .spinner {
354   border: 3px solid var(--color-surface);
355   border-top-color: var(--color-primary);
356   border-radius: 50%;
357   width: 40px;
358   height: 40px;
359   animation: spin 0.8s linear infinite;

```

```

360 }
361
362 @keyframes spin {
363   to {
364     transform: rotate(360deg);
365   }
366 }
367
368 /* ===== UTILITY CLASSES ===== */
369 .container {
370   max-width: 1400px;
371   margin: 0 auto;
372   padding: 0 var(--space-lg);
373 }
374
375 .grid {
376   display: grid;
377   gap: var(--space-lg);
378 }
379
380 .grid-2 {
381   grid-template-columns: repeat(auto-fit, minmax(300px, 1fr));
382 }
383
384 .grid-3 {
385   grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
386 }
387
388 .flex {
389   display: flex;
390 }
391
392 .flex-center {
393   display: flex;
394   align-items: center;
395   justify-content: center;
396 }
397
398 .flex-between {
399   display: flex;
400   align-items: center;
401   justify-content: space-between;
402 }
403
404 .text-center {
405   text-align: center;
406 }
407
408 .text-success {
409   color: var(--color-success);
410 }
411
412 .text-danger {
413   color: var(--color-danger);
414 }
415
416 .text-muted {
417   color: var(--color-text-muted);
418 }
419
420 .mt-1 { margin-top: var(--space-sm); }
421 .mt-2 { margin-top: var(--space-md); }
422 .mt-3 { margin-top: var(--space-lg); }
423 .mt-4 { margin-top: var(--space-xl); }
424
425 .mb-1 { margin-bottom: var(--space-sm); }
426 .mb-2 { margin-bottom: var(--space-md); }
427 .mb-3 { margin-bottom: var(--space-lg); }
428 .mb-4 { margin-bottom: var(--space-xl); }
429
430 .p-1 { padding: var(--space-sm); }
431 .p-2 { padding: var(--space-md); }

```

```

432 .p-3 { padding: var(--space-lg); }
433 .p-4 { padding: var(--space-xl); }
434
435 /* ===== RESPONSIVE DESIGN ===== */
436 @media (max-width: 768px) {
437   :root {
438     --font-size-4xl: 1.875rem;
439     --font-size-3xl: 1.5rem;
440     --font-size-2xl: 1.25rem;
441   }
442
443   .grid-2,
444   .grid-3 {
445     grid-template-columns: 1fr;
446   }
447
448   .container {
449     padding: 0 var(--space-md);
450   }
451 }
452
453 /* ===== SCROLLBAR STYLING ===== */
454 ::-webkit-scrollbar {
455   width: 8px;
456   height: 8px;
457 }
458
459 ::-webkit-scrollbar-track {
460   background: var(--color-bg-secondary);
461 }
462
463 ::-webkit-scrollbar-thumb {
464   background: var(--color-surface);
465   border-radius: var(--radius-full);
466 }
467
468 ::-webkit-scrollbar-thumb:hover {
469   background: var(--color-surface-hover);
470 }
471

```